

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

import os
```

```
import pandas as pd
import numpy as np
import scipy as sc
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split
import pickle
import matplotlib.pyplot as plt
from scipy import stats
import tensorflow as tf
from pylab import rcParams
from keras.models import Model, load_model
from keras.layers import Input, Dense
from keras.callbacks import ModelCheckpoint, TensorBoard
from keras import regularizers
%matplotlib inline
sns.set(style='whitegrid', palette='muted', font_scale=1.5)
rcParams['figure.figsize'] = 14, 8
RANDOM_SEED = 42

LABELS = ["Normal", "Fraud"]

# Fix: Use ydata-profiling (the renamed successor of pandas-profiling)
# with explicit version pinning to avoid numba conflicts
print("Installing ydata-profiling (modern replacement for pandas-profiling)...")

# Install specific versions known to work together
# Attempting installation without version pinning to allow pip to find a compatible version.
!pip install -q ydata-profiling

# Restart is NOT needed with these compatible versions
import ydata_profiling as profile
print("ydata-profiling imported successfully!")
```

```
Installing ydata-profiling (modern replacement for pandas-profiling)...
ydata-profiling imported successfully!
```

```
# Load Train Dataset

Train=pd.read_csv("/content/Train-1542865627584.csv")
Train_Beneficiarydata=pd.read_csv("/content/Train_Beneficiarydata-1542865627584.csv")
Train_Inpatientdata=pd.read_csv("/content/Train_Inpatientdata-1542865627584.csv")
Train_Outpatientdata=pd.read_csv("/content/Train_Outpatientdata-1542865627584.csv")

# Load Test Dataset

Test=pd.read_csv("/content/Test-1542969243754.csv")
Test_Beneficiarydata=pd.read_csv("/content/Test_Beneficiarydata-1542969243754.csv")
Test_Inpatientdata=pd.read_csv("/content/Test_Inpatientdata-1542969243754.csv")
Test_Outpatientdata=pd.read_csv("/content/Test_Outpatientdata-1542969243754.csv")
```

```
# Generate a ydata-profiling report for the 'Train' DataFrame and other DataFrames
# You can repeat this for other DataFrames as needed (e.g., Train_Beneficiarydata, Train_Inpatientdata, etc.)

print("Generating profiling report for 'Train' DataFrame...")
```

```

profile_train = profile.ProfileReport(Train, title="Train DataFrame Profile")
profile_train.to_file("train_data_profile.html")
print("Profiling report for 'Train' DataFrame saved to train_data_profile.html")

print("\nGenerating profiling report for 'Train_Beneficiarydata' DataFrame...")
profile_train_beneficiary = profile.ProfileReport(Train_Beneficiarydata, title="Train Beneficiary Data Profile")
profile_train_beneficiary.to_file("train_beneficiary_data_profile.html")
print("Profiling report for 'Train_Beneficiarydata' DataFrame saved to train_beneficiary_data_profile.html")

print("\nGenerating profiling report for 'Train_Inpatientdata' DataFrame...")
profile_train_inpatient = profile.ProfileReport(Train_Inpatientdata, title="Train Inpatient Data Profile")
profile_train_inpatient.to_file("train_inpatient_data_profile.html")
print("Profiling report for 'Train_Inpatientdata' DataFrame saved to train_inpatient_data_profile.html")

print("\nGenerating profiling report for 'Train_Outpatientdata' DataFrame...")
profile_train_outpatient = profile.ProfileReport(Train_Outpatientdata, title="Train Outpatient Data Profile")
profile_train_outpatient.to_file("train_outpatient_data_profile.html")
print("Profiling report for 'Train_Outpatientdata' DataFrame saved to train_outpatient_data_profile.html")

print("\nGenerating profiling report for 'Test' DataFrame...")
profile_test = profile.ProfileReport(Test, title="Test DataFrame Profile")
profile_test.to_file("test_data_profile.html")
print("Profiling report for 'Test' DataFrame saved to test_data_profile.html")

print("\nGenerating profiling report for 'Test_Beneficiarydata' DataFrame...")
profile_test_beneficiary = profile.ProfileReport(Test_Beneficiarydata, title="Test Beneficiary Data Profile")
profile_test_beneficiary.to_file("test_beneficiary_data_profile.html")
print("Profiling report for 'Test_Beneficiarydata' DataFrame saved to test_beneficiary_data_profile.html")

print("\nGenerating profiling report for 'Test_Inpatientdata' DataFrame...")
profile_test_inpatient = profile.ProfileReport(Test_Inpatientdata, title="Test Inpatient Data Profile")
profile_test_inpatient.to_file("test_inpatient_data_profile.html")
print("Profiling report for 'Test_Inpatientdata' DataFrame saved to test_inpatient_data_profile.html")

print("\nGenerating profiling report for 'Test_Outpatientdata' DataFrame...")
profile_test_outpatient = profile.ProfileReport(Test_Outpatientdata, title="Test Outpatient Data Profile")
profile_test_outpatient.to_file("test_outpatient_data_profile.html")
print("Profiling report for 'Test_Outpatientdata' DataFrame saved to test_outpatient_data_profile.html")

```


Generating profiling report for 'Train' DataFrame...

Summarize dataset: 100%

11/11 [00:01<00:00, 6.45it/s, Completed]

0%| | 0/2 [00:00<?, ?it/s]
100%| | 2/2 [00:00<00:00, 2.41it/s]

Generate report structure: 100%

1/1 [00:01<00:00, 1.71s/it]

Render HTML: 100%

1/1 [00:01<00:00, 1.24s/it]

Export report to file: 100%

1/1 [00:00<00:00, 13.17it/s]

Profiling report for 'Train' DataFrame saved to train_data_profile.html

Generating profiling report for 'Train_Beneficiarydata' DataFrame...

Summarize dataset: 100%

98/98 [00:47<00:00, 1.72it/s, Completed]

0%| | 0/25 [00:00<?, ?it/s]
4%| | 1/25 [00:04<01:58, 4.92s/it]
8%| | 2/25 [00:05<00:49, 2.14s/it]
12%| | 3/25 [00:05<00:28, 1.29s/it]
16%| | 4/25 [00:05<00:17, 1.20it/s]
24%| | 6/25 [00:06<00:13, 1.38it/s]
28%| | 7/25 [00:06<00:09, 1.81it/s]
44%| | 11/25 [00:07<00:04, 3.45it/s]
48%| | 12/25 [00:07<00:03, 3.83it/s]
52%| | 13/25 [00:07<00:03, 3.61it/s]
56%| | 14/25 [00:08<00:03, 3.64it/s]
60%| | 15/25 [00:08<00:02, 3.73it/s]
64%| | 16/25 [00:08<00:02, 4.11it/s]
68%| | 17/25 [00:08<00:02, 3.63it/s]
72%| | 18/25 [00:09<00:01, 4.15it/s]
76%| | 19/25 [00:09<00:01, 3.72it/s]
80%| | 20/25 [00:09<00:01, 4.39it/s]
84%| | 21/25 [00:09<00:00, 4.85it/s]
100%| | 25/25 [00:09<00:00, 2.51it/s]

Generate report structure: 100%

1/1 [00:07<00:00, 7.15s/it]

Render HTML: 100%

1/1 [00:00<00:00, 2.25it/s]

Export report to file: 100%

1/1 [00:00<00:00, 23.41it/s]

Profiling report for 'Train_Beneficiarydata' DataFrame saved to train_beneficiary_data_profile.html

Generating profiling report for 'Train_Inpatientdata' DataFrame...

Summarize dataset: 100%

76/76 [00:10<00:00, 5.95it/s, Completed]

0%| | 0/30 [00:00<?, ?it/s]
3%| | 1/30 [00:01<00:40, 1.39s/it]
10%| | 3/30 [00:01<00:13, 2.02it/s]
17%| | 5/30 [00:02<00:08, 2.90it/s]
23%| | 7/30 [00:02<00:06, 3.69it/s]
27%| | 8/30 [00:02<00:05, 3.96it/s]
30%| | 9/30 [00:02<00:04, 4.40it/s]
33%| | 10/30 [00:03<00:04, 4.46it/s]
37%| | 11/30 [00:03<00:03, 4.77it/s]
40%| | 12/30 [00:03<00:03, 4.94it/s]
43%| | 13/30 [00:03<00:03, 4.69it/s]
53%| | 16/30 [00:03<00:02, 6.71it/s]
57%| | 17/30 [00:04<00:01, 6.76it/s]
60%| | 18/30 [00:04<00:01, 6.60it/s]
63%| | 19/30 [00:04<00:01, 6.97it/s]
67%| | 20/30 [00:04<00:01, 6.55it/s]
70%| | 21/30 [00:04<00:01, 7.12it/s]
73%| | 22/30 [00:04<00:01, 6.39it/s]
80%| | 24/30 [00:04<00:00, 8.45it/s]
100%| | 30/30 [00:05<00:00, 5.87it/s]

Generate report structure: 100%

1/1 [00:20<00:00, 20.77s/it]

Render HTML: 100%

1/1 [00:00<00:00, 1.70it/s]

Export report to file: 100%

1/1 [00:00<00:00, 14.98it/s]

Profiling report for 'Train_Inpatientdata' DataFrame saved to train_inpatient_data_profile.html

Generating profiling report for 'Train_Outpatientdata' DataFrame...

Summarize dataset: 100%

53/53 [00:49<00:00, 1.15s/it, Completed]

0%| | 0/27 [00:00<?, ?it/s]
4%| | 1/27 [00:05<02:18, 5.32s/it]
7%| | 2/27 [00:09<01:55, 4.61s/it]
11%| | 3/27 [00:10<01:09, 2.90s/it]
15%| | 4/27 [00:12<00:58, 2.52s/it]

19%|██████| 5/27 [00:12<00:36, 1.66s/it]
22%|██████| 6/27 [00:12<00:25, 1.20s/it]
26%|██████| 7/27 [00:17<00:49, 2.45s/it]
30%|██████| 8/27 [00:17<00:33, 1.75s/it]
33%|██████| 9/27 [00:20<00:34, 1.89s/it]
37%|██████| 10/27 [00:20<00:22, 1.35s/it]
41%|██████| 11/27 [00:23<00:31, 1.95s/it]
44%|██████| 12/27 [00:23<00:21, 1.44s/it]
48%|██████| 13/27 [00:25<00:21, 1.53s/it]
52%|██████| 14/27 [00:25<00:15, 1.16s/it]
56%|██████| 15/27 [00:27<00:17, 1.42s/it]
63%|██████| 17/27 [00:32<00:17, 1.77s/it]
67%|██████| 18/27 [00:32<00:12, 1.36s/it]
70%|██████| 19/27 [00:33<00:09, 1.23s/it]
81%|██████| 22/27 [00:36<00:05, 1.07s/it]
96%|██████| 26/27 [00:36<00:00, 1.63it/s]
100%|██████| 27/27 [00:37<00:00, 1.39s/it]

Generate report structure: 100%

1/1 [00:19<00:00, 19.62s/it]

Render HTML: 100%

1/1 [00:00<00:00, 2.38it/s]

Export report to file: 100%

1/1 [00:00<00:00, 14.64it/s]

Profiling report for 'Train_Outpatientdata' DataFrame saved to train_outpatient_data_profile.html

Generating profiling report for 'Test' DataFrame...

Summarize dataset: 100%

10/10 [00:00<00:00, 26.97it/s, Completed]

100%|██████| 1/1 [00:00<00:00, 27.45it/s]

Generate report structure: 100%

1/1 [00:00<00:00, 1.14it/s]

Render HTML: 100%

1/1 [00:00<00:00, 21.17it/s]

Export report to file: 100%

1/1 [00:00<00:00, 62.44it/s]

Profiling report for 'Test' DataFrame saved to test_data_profile.html

Generating profiling report for 'Test_Beneficiarydata' DataFrame...

Summarize dataset: 100%

98/98 [00:15<00:00, 3.33it/s, Completed]

0%|██████| 0/25 [00:00<?, ?it/s]
4%|██████| 1/25 [00:00<00:13, 1.73it/s]
20%|██████| 5/25 [00:00<00:02, 7.80it/s]
28%|██████| 7/25 [00:00<00:01, 9.16it/s]
40%|██████| 10/25 [00:01<00:01, 11.91it/s]
56%|██████| 14/25 [00:01<00:00, 14.20it/s]
68%|██████| 17/25 [00:01<00:00, 14.70it/s]
76%|██████| 19/25 [00:01<00:00, 14.85it/s]
100%|██████| 25/25 [00:01<00:00, 13.62it/s]

Generate report structure: 100%

1/1 [00:07<00:00, 7.36s/it]

Render HTML: 100%

1/1 [00:00<00:00, 2.03it/s]

Export report to file: 100%

1/1 [00:00<00:00, 23.49it/s]

Profiling report for 'Test_Beneficiarydata' DataFrame saved to test_beneficiary_data_profile.html

Generating profiling report for 'Test_Inpatientdata' DataFrame...

Summarize dataset: 100%

65/65 [00:05<00:00, 4.51it/s, Completed]

0%|██████| 0/30 [00:00<?, ?it/s]
3%|██████| 1/30 [00:00<00:04, 6.00it/s]
10%|██████| 3/30 [00:00<00:02, 11.23it/s]
17%|██████| 5/30 [00:00<00:01, 14.44it/s]
23%|██████| 7/30 [00:00<00:01, 13.89it/s]
30%|██████| 9/30 [00:00<00:01, 15.24it/s]
40%|██████| 12/30 [00:00<00:01, 16.28it/s]
47%|██████| 14/30 [00:00<00:00, 17.00it/s]
53%|██████| 16/30 [00:01<00:00, 15.25it/s]
60%|██████| 18/30 [00:01<00:00, 15.18it/s]
67%|██████| 20/30 [00:01<00:00, 15.33it/s]
73%|██████| 22/30 [00:01<00:00, 14.98it/s]
80%|██████| 24/30 [00:01<00:00, 16.02it/s]
100%|██████| 30/30 [00:01<00:00, 17.35it/s]

Generate report structure: 100%

1/1 [00:21<00:00, 21.17s/it]

Render HTML: 100%

1/1 [00:00<00:00, 2.23it/s]

Export report to file: 100%

1/1 [00:00<00:00, 18.07it/s]

Profiling report for 'Test_Inpatientdata' DataFrame saved to test_inpatient_data_profile.html

Generating profiling report for 'Test_Outpatientdata' DataFrame...

Summarize dataset: 100%

46/46 [00:13<00:00, 3.09it/s, Completed]

```
## Lets Check Shape of datasets
```

```
print('Shape of Train data :',Train.shape)
print('Shape of Train_Beneficiarydata data :',Train_Beneficiarydata.shape)
print('Shape of Train_Inpatientdata data :',Train_Inpatientdata.shape)
print('Shape of Train_Outpatientdata data :',Train_Outpatientdata.shape)

print('Shape of Test data :',Test.shape)
print('Shape of Test_Beneficiarydata data :',Test_Beneficiarydata.shape)
print('Shape of Test_Inpatientdata data :',Test_Inpatientdata.shape)
print('Shape of Test_Outpatientdata data :',Test_Outpatientdata.shape)
```

```
Shape of Train data : (5410, 2) 17/27 [00:00:03, 3.26it/s]
Shape of Train_Beneficiarydata data : (1385, 125) 18/27 [00:00:02, 2.13it/s]
Shape of Train_Inpatientdata data : (4047, 30) 20/27 [00:00:02, 4.04it/s]
Shape of Train_Outpatientdata data : (5173, 27) 22/27 [00:00:01, 5.17it/s]
Shape of Test data : (1353, 1) 27/27 [00:00:00, 2.99it/s]
Shape of Test_Beneficiarydata data : (63968, 25) 1/1 [00:16<00:00, 16.76s/it]
Shape of Test_Inpatientdata data : (9551, 30)
Shape of Test_Outpatientdata data : (125841, 27) 1/1 [00:00<00:00, 2.82it/s]
```

Export report to file: 100%

1/1 [00:00<00:00, 14.71it/s]

Basic Data Understanding

Profiling report for 'Test_Outpatientdata' DataFrame saved to test_outpatient_data_profile.html

Train and Test Data Understanding

```
## Lets check shape of Train and Test Data
```

```
print('Train Shape:',Train.shape,'\n')
print('Train Sample:\n',Train.head(2),'\n')

print('\n Test Shape:',Test.shape,'\n')
print('Test Sample: \n',Test.head(2))
```

Train Shape: (5410, 2)

Train Sample:

	Provider	PotentialFraud
0	PRV51001	No
1	PRV51003	Yes

Test Shape: (1353, 1)

Test Sample:

	Provider
0	PRV51002
1	PRV51006

```
## Lets check whether providers details are unique or not in train data
```

```
print(Train.Provider.value_counts(sort=True,ascending=False).head(2)) # number of unique providers in train data.Check for dup
```

```
print('\n Total missing values in Train :',Train.isna().sum().sum())
```

```
print('\n Total missing values in Train :',Test.isna().sum().sum())
```

```
Provider
PRV57763    1
PRV51001    1
Name: count, dtype: int64
```

Total missing values in Train : 0

Total missing values in Train : 0

Beneficiary Data Understanding

```
#Lets Check missing values in each column in beneficiary data :
```

```
Train_Beneficiarydata.isna().sum()
Test_Beneficiarydata.isna().sum()
```

	0
BenelID	0
DOB	0
DOD	63394
Gender	0
Race	0
RenalDiseaseIndicator	0
State	0
County	0
NoOfMonths_PartACov	0
NoOfMonths_PartBCov	0
ChronicCond_Alzheimer	0
ChronicCond_Heartfailure	0
ChronicCond_KidneyDisease	0
ChronicCond_Cancer	0
ChronicCond_ObstrPulmonary	0
ChronicCond_Depression	0
ChronicCond_Diabetes	0
ChronicCond_IschemicHeart	0
ChronicCond_Osteoporasis	0
ChronicCond_rheumatoidarthritis	0
ChronicCond_stroke	0
IPAnnualReimbursementAmt	0
IPAnnualDeductibleAmt	0
OPAnnualReimbursementAmt	0
OPAnnualDeductibleAmt	0

dtype: int64

```
# Lets check data types of each column in beneficiary data

Train_Beneficiarydata.dtypes
Test_Beneficiarydata.dtypes
```

	0
BenelID	object
DOB	object
DOD	object
Gender	int64
Race	int64
RenalDiseaseIndicator	object
State	int64
County	int64
NoOfMonths_PartACov	int64
NoOfMonths_PartBCov	int64
ChronicCond_Alzheimer	int64
ChronicCond_Heartfailure	int64
ChronicCond_KidneyDisease	int64
ChronicCond_Cancer	int64
ChronicCond_ObstrPulmonary	int64
ChronicCond_Depression	int64
ChronicCond_Diabetes	int64
ChronicCond_IschemicHeart	int64
ChronicCond_Osteoporasis	int64
ChronicCond_rheumatoidarthritis	int64
ChronicCond_stroke	int64
IPAnnualReimbursementAmt	int64
IPAnnualDeductibleAmt	int64
OPAnnualReimbursementAmt	int64
OPAnnualDeductibleAmt	int64

dtype: object

```
##Replacing 2 with 0 for chronic conditions ,that means chroniv condition No is 0 and yes is 1

Train_Beneficiarydata = Train_Beneficiarydata.replace({'ChronicCond_Alzheimer': 2, 'ChronicCond_Heartfailure': 2, 'ChronicCond_
    'ChronicCond_Cancer': 2, 'ChronicCond_ObstrPulmonary': 2, 'ChronicCond_Depression': 2,
    'ChronicCond_Diabetes': 2, 'ChronicCond_IschemicHeart': 2, 'ChronicCond_Osteoporasis': 2,
    'ChronicCond_rheumatoidarthritis': 2, 'ChronicCond_stroke': 2 }, 0)

Train_Beneficiarydata = Train_Beneficiarydata.replace({'RenalDiseaseIndicator': 'Y'}, 1)

Test_Beneficiarydata = Test_Beneficiarydata.replace({'ChronicCond_Alzheimer': 2, 'ChronicCond_Heartfailure': 2, 'ChronicCond_Ki
    'ChronicCond_Cancer': 2, 'ChronicCond_ObstrPulmonary': 2, 'ChronicCond_Depression': 2,
    'ChronicCond_Diabetes': 2, 'ChronicCond_IschemicHeart': 2, 'ChronicCond_Osteoporasis': 2,
    'ChronicCond_rheumatoidarthritis': 2, 'ChronicCond_stroke': 2 }, 0)

Test_Beneficiarydata = Test_Beneficiarydata.replace({'RenalDiseaseIndicator': 'Y'}, 1)
```



```
Train_Beneficiarydata.head(1)
Test_Beneficiarydata.head(1)
```

	BeneID	DOB	DOD	Gender	Race	RenalDiseaseIndicator	State	County	NoOfMonths_PartACov	NoOfMonths_PartBCov	...	Chr
0	BENE11001	1943-01-01	NaN	1	1		0	39	230	12	12	...

1 rows × 25 columns

Add Age of Person based on his/her DOD(Date of death) and DOB (Date of Birth)

```
## Lets Create Age column to the dataset
```

```
Train_Beneficiarydata['DOB'] = pd.to_datetime(Train_Beneficiarydata['DOB'], format = '%Y-%m-%d')
Train_Beneficiarydata['DOD'] = pd.to_datetime(Train_Beneficiarydata['DOD'], format = '%Y-%m-%d', errors='ignore')
Train_Beneficiarydata['Age'] = round(((Train_Beneficiarydata['DOD'] - Train_Beneficiarydata['DOB']).dt.days)/365)
```

```
Test_Beneficiarydata['DOB'] = pd.to_datetime(Test_Beneficiarydata['DOB'], format = '%Y-%m-%d')
Test_Beneficiarydata['DOD'] = pd.to_datetime(Test_Beneficiarydata['DOD'], format = '%Y-%m-%d', errors='ignore')
Test_Beneficiarydata['Age'] = round(((Test_Beneficiarydata['DOD'] - Test_Beneficiarydata['DOB']).dt.days)/365)
```

```
## As we see that last DOD value is 2009-12-01 ,which means Beneficiary Details data is of year 2009.
## so we will calculate age of other beneficiaries for year 2009.
```

```
Train_Beneficiarydata.Age.fillna(round(((pd.to_datetime('2009-12-01', format = '%Y-%m-%d') - Train_Beneficiarydata['DOB']).dt.days)/365), inplace=True)
```

```
Test_Beneficiarydata.Age.fillna(round(((pd.to_datetime('2009-12-01', format = '%Y-%m-%d') - Test_Beneficiarydata['DOB']).dt.days)/365), inplace=True)
```

```
Train_Beneficiarydata.head(2)
Test_Beneficiarydata.head(2)
```

	BeneID	DOB	DOD	Gender	Race	RenalDiseaseIndicator	State	County	NoOfMonths_PartACov	NoOfMonths_PartBCov	...	Chr
0	BENE11001	1943-01-01	NaT	1	1		0	39	230	12	12	...
1	BENE11007	1940-09-01	2009-12-01	1	2		0	45	610	12	12	...

2 rows × 26 columns

Add Flag column 'WhetherDead' using DOD values to tell whether beneficiary is dead or not

```
#Lets create a new variable 'WhetherDead' with flag 1 means Dead and 0 means not Dead
```

```
Train_Beneficiarydata.loc[Train_Beneficiarydata.DOD.isna(), 'WhetherDead'] = 0
Train_Beneficiarydata.loc[Train_Beneficiarydata.DOD.notna(), 'WhetherDead'] = 1
Train_Beneficiarydata.loc[:, 'WhetherDead'].head(7)
```

```
Test_Beneficiarydata.loc[Test_Beneficiarydata.DOD.isna(), 'WhetherDead'] = 0
Test_Beneficiarydata.loc[Test_Beneficiarydata.DOD.notna(), 'WhetherDead'] = 1
Test_Beneficiarydata.loc[:, 'WhetherDead'].head(3)
```

```
WhetherDead
0          0.0
1          1.0
2          0.0

dtype: float64
```

```
#Lets check missing values in each column in inpatient data
Train_Inpatientdata.isna().sum()

Test_Inpatientdata.isna().sum()
```

```

0
BenelD          0
ClaimID         0
ClaimStartDt    0
ClaimEndDt      0
Provider        0
InscClaimAmtReimbursed  0
AttendingPhysician    31
OperatingPhysician    3962
OtherPhysician        8538
AdmissionDt          0
ClmAdmitDiagnosisCode  0
DeductibleAmtPaid    196
DischargeDt         0
DiagnosisGroupCode   0
ClmDiagnosisCode_1   0
ClmDiagnosisCode_2   54
ClmDiagnosisCode_3   169
ClmDiagnosisCode_4   404
ClmDiagnosisCode_5   719
ClmDiagnosisCode_6  1197
ClmDiagnosisCode_7  1736
ClmDiagnosisCode_8  2360
ClmDiagnosisCode_9  3238
ClmDiagnosisCode_10 8664
ClmProcedureCode_1   4118
ClmProcedureCode_2   8297
ClmProcedureCode_3   9328
ClmProcedureCode_4   9522
ClmProcedureCode_5   9549
ClmProcedureCode_6   9551

dtype: int64
```

Create new column 'AdmitForDays' indicating number of days patient was admitted in hospita

```
## As patient can be admitted for only for 1 day,we will add 1 to the difference of Discharge Date and Admission Date

Train_Inpatientdata['AdmissionDt'] = pd.to_datetime(Train_Inpatientdata['AdmissionDt'] , format = '%Y-%m-%d')
Train_Inpatientdata['DischargeDt'] = pd.to_datetime(Train_Inpatientdata['DischargeDt'],format = '%Y-%m-%d')
```

```
Train_Inpatientdata['AdmitForDays'] = ((Train_Inpatientdata['DischargeDt'] - Train_Inpatientdata['AdmissionDt']).dt.days)+1

Test_Inpatientdata['AdmissionDt'] = pd.to_datetime(Test_Inpatientdata['AdmissionDt'], format = '%Y-%m-%d')
Test_Inpatientdata['DischargeDt'] = pd.to_datetime(Test_Inpatientdata['DischargeDt'], format = '%Y-%m-%d')
Test_Inpatientdata['AdmitForDays'] = ((Test_Inpatientdata['DischargeDt'] - Test_Inpatientdata['AdmissionDt']).dt.days)+1
```

```
## Lets check Min and Max values of AdmitForDays column in Train and Test.
print('Min AdmitForDays Train:- ', Train_Inpatientdata.AdmitForDays.min())
print('Max AdmitForDays Train:- ', Train_Inpatientdata.AdmitForDays.max())
Train_Inpatientdata.AdmitForDays.isnull().sum() #Check Null values.

print('Min AdmitForDays Test:- ', Test_Inpatientdata.AdmitForDays.min())
print('Max AdmitForDays Test:- ', Test_Inpatientdata.AdmitForDays.max())
Test_Inpatientdata.AdmitForDays.isnull().sum() #Check Null values.
```

```
Min AdmitForDays Train:- 1
Max AdmitForDays Train:- 36
Min AdmitForDays Test:- 1
Max AdmitForDays Test:- 36
np.int64(0)
```

Outpatient Data understanding

Lets Check null values in each column of outpatient data

```
Train_Outpatientdata.isna().sum()

Test_Outpatientdata.isna().sum()
```

	0
BenelID	0
ClaimID	0
ClaimStartDt	0
ClaimEndDt	0
Provider	0
InscClaimAmtReimbursed	0
AttendingPhysician	316
OperatingPhysician	104237
OtherPhysician	78222
CImDiagnosisCode_1	2578
CImDiagnosisCode_2	47731
CImDiagnosisCode_3	76575
CImDiagnosisCode_4	95371
CImDiagnosisCode_5	107875
CImDiagnosisCode_6	114035
CImDiagnosisCode_7	117871
CImDiagnosisCode_8	120310
CImDiagnosisCode_9	122278
CImDiagnosisCode_10	125578
CImProcedureCode_1	125807
CImProcedureCode_2	125832
CImProcedureCode_3	125839
CImProcedureCode_4	125841
CImProcedureCode_5	125841
CImProcedureCode_6	125841
DeductibleAmtPaid	0
CImAdmitDiagnosisCode	100036

dtype: int64

```
## Lets Check Shape of datasets after adding new variables

print('Shape of Train data :',Train.shape)
print('Shape of Train_Beneficiarydata data :',Train_Beneficiarydata.shape)
print('Shape of Train_Inpatientdata data :',Train_Inpatientdata.shape)
print('Shape of Train_Outpatientdata data :',Train_Outpatientdata.shape)

print('Shape of Test data :',Test.shape)
print('Shape of Test_Beneficiarydata data :',Test_Beneficiarydata.shape)
print('Shape of Test_Inpatientdata data :',Test_Inpatientdata.shape)
print('Shape of Test_Outpatientdata data :',Test_Outpatientdata.shape)
```

Shape of Train data : (5410, 2)
Shape of Train_Beneficiarydata data : (138556, 27)
Shape of Train_Inpatientdata data : (40474, 31)
Shape of Train_Outpatientdata data : (517737, 27)
Shape of Test data : (1353, 1)
Shape of Test_Beneficiarydata data : (63968, 27)
Shape of Test_Inpatientdata data : (9551, 31)
Shape of Test_Outpatientdata data : (125841, 27)

Data Merging

As we see columns in inpatient and outpatient data are similar, we will merge this data based on these similar keys using outer join.

```
Key_Column_To_Merge_Outpatient=Train_Outpatientdata.columns
print(Key_Column_To_Merge_Outpatient)
```

```
Index(['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
       'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
       'OtherPhysician', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
       'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
       'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
       'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
       'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
       'ClmProcedureCode_5', 'ClmProcedureCode_6', 'DeductibleAmtPaid',
       'ClmAdmitDiagnosisCode'],
      dtype='object')
```

****Merge Inpatient and Outpatient data and create dataset for all patients. ****

```
# Lets make union of Inpatient and outpatient data .
# We will use all keys in outpatient data as we want to make union and dont want duplicate columns from both tables.
```

```
Train_Allpatientdata=pd.merge(Train_Outpatientdata,Train_Inpatientdata,
                              left_on=['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
                                         'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
                                         'OtherPhysician', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
                                         'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
                                         'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
                                         'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
                                         'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                                         'ClmProcedureCode_5', 'ClmProcedureCode_6', 'DeductibleAmtPaid',
                                         'ClmAdmitDiagnosisCode'],
                              right_on=['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
                                         'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
                                         'OtherPhysician', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
                                         'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
                                         'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
                                         'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
                                         'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                                         'ClmProcedureCode_5', 'ClmProcedureCode_6', 'DeductibleAmtPaid',
                                         'ClmAdmitDiagnosisCode'],
                              how='outer')
```

```
Test_Allpatientdata=pd.merge(Test_Outpatientdata,Test_Inpatientdata,
                              left_on=['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
                                         'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
                                         'OtherPhysician', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
                                         'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
                                         'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
                                         'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
                                         'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                                         'ClmProcedureCode_5', 'ClmProcedureCode_6', 'DeductibleAmtPaid',
                                         'ClmAdmitDiagnosisCode'],
                              right_on=['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
                                         'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
                                         'OtherPhysician', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
                                         'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
                                         'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
                                         'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
                                         'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                                         'ClmProcedureCode_5', 'ClmProcedureCode_6', 'DeductibleAmtPaid',
                                         'ClmAdmitDiagnosisCode'],
                              how='outer')
```

```
print('Train_Allpatientdata',Train_Allpatientdata.shape)
print('Test_Allpatientdata',Test_Allpatientdata.shape)
```

```
Train_Allpatientdata (558211, 31)
Test_Allpatientdata (135392, 31)
```

```
Train_Allpatientdata.head(4)
Test_Allpatientdata.head(4)
```

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician
0	BENE100001	CLM129501	2009-01-10	2009-01-10	PRV52050	1100	PHY364771	NaN
1	BENE100002	CLM623189	2009-10-10	2009-10-10	PRV52083	60	PHY402651	NaN
2	BENE100004	CLM546697	2009-08-26	2009-09-15	PRV53275	1500	PHY364921	NaN
3	BENE100004	CLM63603	2009-08-12	2009-08-17	PRV52959	11000	PHY410241	NaN

4 rows × 31 columns

Merge Beneficiary details to All Patients data

```
## Lets merge All patient data with beneficiary details data based on 'BeneID' as joining key for inner join
Train_AllPatientDetailsdata=pd.merge(Train_Allpatientdata,Train_Beneficiarydata,left_on='BeneID',right_on='BeneID',how='inner')

Test_AllPatientDetailsdata=pd.merge(Test_Allpatientdata,Test_Beneficiarydata,left_on='BeneID',right_on='BeneID',how='inner')
```

```
## Lets Print shape of data
print('Shape of All Patient Details Train : ',Train_AllPatientDetailsdata.shape)
Train_AllPatientDetailsdata.head()

print('Shape of All Patient Details Test : ',Test_AllPatientDetailsdata.shape)
Test_AllPatientDetailsdata.head(2)
```

```
Shape of All Patient Details Train : (558211, 57)
Shape of All Patient Details Test : (135392, 57)
```

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician
0	BENE100001	CLM129501	2009-01-10	2009-01-10	PRV52050	1100	PHY364771	NaN
1	BENE100002	CLM623189	2009-10-10	2009-10-10	PRV52083	60	PHY402651	NaN

2 rows × 57 columns

Merge PotentialFraud details for each provider to create ProviderWithPatientDetails data

```
# Lets merge patient data with fradulent providers details data with "Provider" as joining key for inner join

Train_ProviderWithPatientDetailsdata=pd.merge(Train,Train_AllPatientDetailsdata,on='Provider')

Test_ProviderWithPatientDetailsdata=pd.merge(Test,Test_AllPatientDetailsdata,on='Provider')
```

```
## Lets Print the shape of dataset
print("Shape of Train Provider with Patient Details data :-",Train_ProviderWithPatientDetailsdata.shape)
Train_ProviderWithPatientDetailsdata.head()

print("Shape of Test Provider with Patient Details data :-",Test_ProviderWithPatientDetailsdata.shape)
Test_ProviderWithPatientDetailsdata.head(2)
```

Shape of Train Provider with Patient Details data :- (558211, 58)

Shape of Test Provider with Patient Details data :- (135392, 57)

	Provider	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician
0	PRV51002	BENE101165	CLM304833	2009-04-15	2009-04-15	70	PHY410547	NaN
1	PRV51002	BENE102518	CLM529049	2009-08-16	2009-08-16	20	PHY315513	PHY418464

2 rows × 57 columns

Lets check missing value percentage in full datasets

```
#print('Percent missing values in Train Provider with patient details data :\n')
```

```
Train_ProviderWithPatientDetailsdata.isnull().sum()*100/len(Train_ProviderWithPatientDetailsdata)
```

```
print('Percent missing values in Test Provider with patient details data :\n')
```

```
Test_ProviderWithPatientDetailsdata.isnull().sum()*100/len(Test_ProviderWithPatientDetailsdata)
```


Percent missing values in Test Provider with patient details data :

	0
Provider	0.000000
BenelID	0.000000
ClaimID	0.000000
ClaimStartDt	0.000000
ClaimEndDt	0.000000
InscClaimAmtReimbursed	0.000000
AttendingPhysician	0.256293
OperatingPhysician	79.915357
OtherPhysician	64.080596
ClmDiagnosisCode_1	1.904101
ClmDiagnosisCode_2	35.293814
ClmDiagnosisCode_3	56.682817

Lets Check dtypes of both the datasets
 Train_ProviderWithPatientDetailsdata.dtypes
 Test_ProviderWithPatientDetailsdata.dtypes

ClmDiagnosisCode_6	85.109903
ClmDiagnosisCode_7	88.341261
ClmDiagnosisCode_8	90.603581
ClmDiagnosisCode_9	92.705625
ClmDiagnosisCode_10	99.150615
ClmProcedureCode_1	95.962095
ClmProcedureCode_2	99.067153
ClmProcedureCode_3	99.833816
ClmProcedureCode_4	99.978581
ClmProcedureCode_5	99.998523
ClmProcedureCode_6	100.000000
DeductibleAmtPaid	0.144765
ClmAdmitDiagnosisCode	73.886197
AdmissionDt	92.945669
DischargeDt	92.945669
DiagnosisGroupCode	92.945669
AdmitForDays	92.945669
DOB	0.000000
DOD	99.231860
Gender	0.000000
Race	0.000000
RenalDiseaseIndicator	0.000000
State	0.000000
County	0.000000
NoOfMonths_PartACov	0.000000
NoOfMonths_PartBCov	0.000000
ChronicCond_Alzheimer	0.000000
ChronicCond_Heartfailure	0.000000
ChronicCond_KidneyDisease	0.000000
ChronicCond_Cancer	0.000000
ChronicCond_ObstrPulmonary	0.000000

```
ChronicCond_Depression      0.000000
ChronicCond_Diabetes         0.000000
ChronicCond_IschemicHeart    0.000000
ChronicCond_Osteoporosis     0.000000
ChronicCond_rheumatoidarthritis 0.000000
ChronicCond_stroke           0.000000
IPAnnualReimbursementAmt     0.000000
IPAnnualDeductibleAmt        0.000000
OPAnnualReimbursementAmt     0.000000
OPAnnualDeductibleAmt        0.000000
Age                           0.000000
WhetherDead                   0.000000
```

dtype: float64

	0
Provider	object
BenelID	object
ClaimID	object
ClaimStartDt	object
ClaimEndDt	object
InscClaimAmtReimbursed	int64
AttendingPhysician	object
OperatingPhysician	object
OtherPhysician	object
ClmDiagnosisCode_1	object
ClmDiagnosisCode_2	object
ClmDiagnosisCode_3	object
ClmDiagnosisCode_4	object
ClmDiagnosisCode_5	object

Exploratory Data Analysis

Plot Potential fraud class proportion in both Train and Merged data

```
#Plotting the frequencies of fraud and non-fraud Merged transactions in the data

sns.set_style('white',rc={'figure.figsize':(12,8)})
count_classes = pd.value_counts(Train_ProviderWithPatientDetailsdata['PotentialFraud'], sort = True)
print("Percent Distribution of Potential Fraud class:- \n",count_classes*100/len(Train_ProviderWithPatientDetailsdata))
LABELS = ["Non Fraud", "Fraud"]
#Drawing a barplot
count_classes.plot(kind = 'bar', rot=0,figsize=(10,6))

#Giving titles and labels to the plot
plt.title("Potential Fraud distribution in Aggregated claim transactional data")
plt.xticks(range(2), LABELS)
plt.xlabel("Potential Fraud Class ")
plt.ylabel("Number of PotentialFraud per Class ")

plt.savefig('PotentialFraudDistributionInMergedData')
```

ClmAdmitDiagnosisCode	object
AdmissionDt	datetime64[ns]
DischargeDt	datetime64[ns]
DiagnosisGroupCode	object
AdmitForDays	float64
DOB	datetime64[ns]
DOD	datetime64[ns]
Gender	int64
Race	int64
RenalDiseaseIndicator	object
State	int64
County	int64
NoOfMonths_PartACov	int64
NoOfMonths_PartBCov	int64
ChronicCond_Alzheimer	int64
ChronicCond_Heartfailure	int64
ChronicCond_KidneyDisease	int64
ChronicCond_Cancer	int64
ChronicCond_ObstrPulmonary	int64
ChronicCond_Depression	int64

```
Percent Distribution of Potential Fraud class:-
PotentialFraud
ChronicCond_Diabetes
No 61.878921
Yes 38.121069
Name: count, dtype: float64
ChronicCond_IschemicHeart
No 61.878921
Yes 38.121069
Name: count, dtype: float64
ChronicCond_Osteoporosis
No 61.878921
Yes 38.121069
Name: count, dtype: float64
```

Potential Fraud distribution in Aggregated claim transactional data



```
#Plotting the frequencies of fraud and non-fraud transactions in the train data

count_classes_provider = pd.value_counts(Train['PotentialFraud'], sort = True)
print("Percent Distribution of Potential Fraud class:- \n",count_classes_provider*100/len(Train))
LABELS = ["Non Fraud", "Fraud"]
#Drawing a barplot
count_classes_provider.plot(kind = 'bar', rot=0,figsize=(10,6))

#Giving titles and labels to the plot
plt.title("Potential Fraud distribution in individual Providers data")
plt.xticks(range(2), LABELS)
plt.xlabel("Potential Fraud Class ")
plt.ylabel("Number of PotentialFraud per Class ")

plt.savefig('PotentialFraudDistributionImbalance')
```

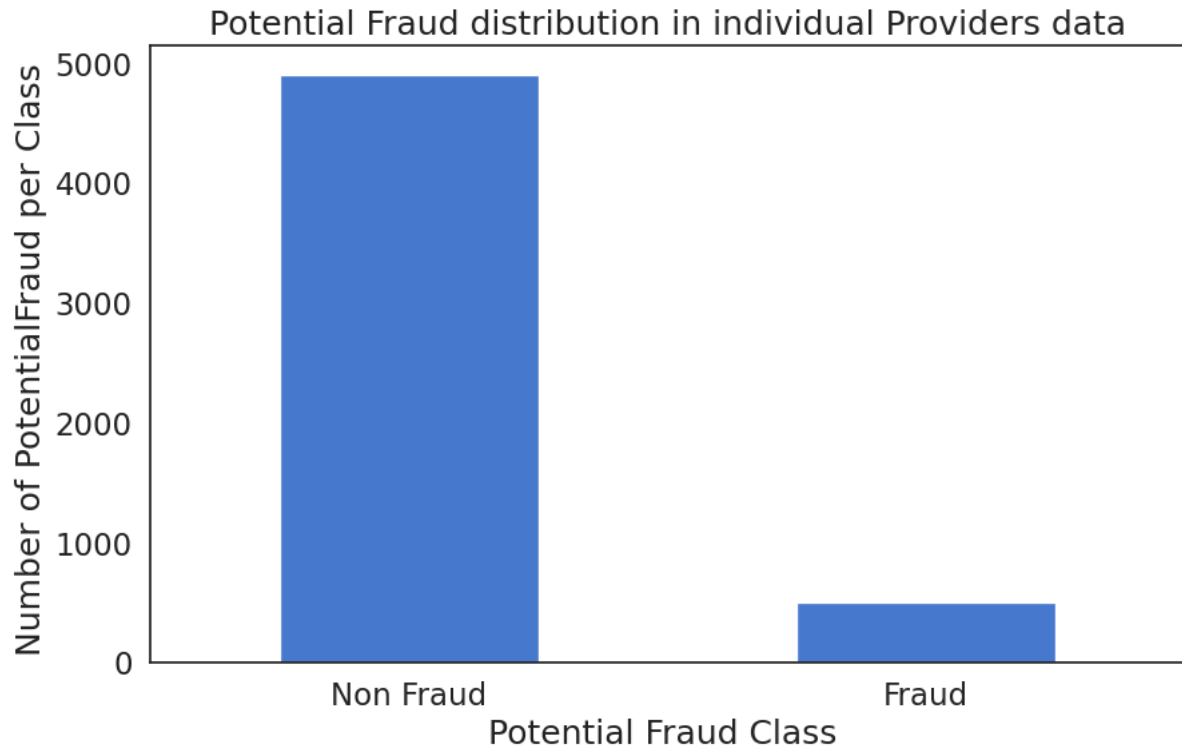
Percent Distribution of Potential Fraud class:-

PotentialFraud

No 90.64695

Yes 9.35305

Name: count, dtype: float64



From the above 2 graphs ,we can say that the proportion of fraudulent claim transactions are more compared to non fraud providers. So we must get insights from number of claim transactions and amounts involved per - Beneficiary | Beneficiary + Physician | Physician | Diagnosis | Procedure etc...

```
print('Providers NonFraud|Fraud Class Percent Distribution in Whole dataset :\n',count_classes/len(Train_ProviderWithPatientDet
```

Providers NonFraud|Fraud Class Percent Distribution in Whole dataset :

PotentialFraud

No 61.878931

Yes 38.121069

Name: count, dtype: float64

```
print('Providers NonFraud|Fraud Class Percent Distribution :\n',count_classes_provider/len(Train)*100)
```

Providers NonFraud|Fraud Class Percent Distribution :

PotentialFraud

No 90.64695

Yes 9.35305

Name: count, dtype: float64

State - wise Percent Beneficiary Distribution

```
#Plotting the frequencies of Statewise beneficiaries
```

```
count_States = pd.value_counts(Train_Beneficiarydata['State'], sort = True)
```

```
#print("Percent Distribution of Beneficiaries per state:- \n",count_States*100/len(Train_Beneficiarydata))
```

```
#Drawing a barplot
```

```
(count_States*100/len(Train_Beneficiarydata)).plot(kind = 'bar', rot=0,figsize=(16,8),fontsize=12,legend=True)
```

```
#Giving titles and labels to the plot
```

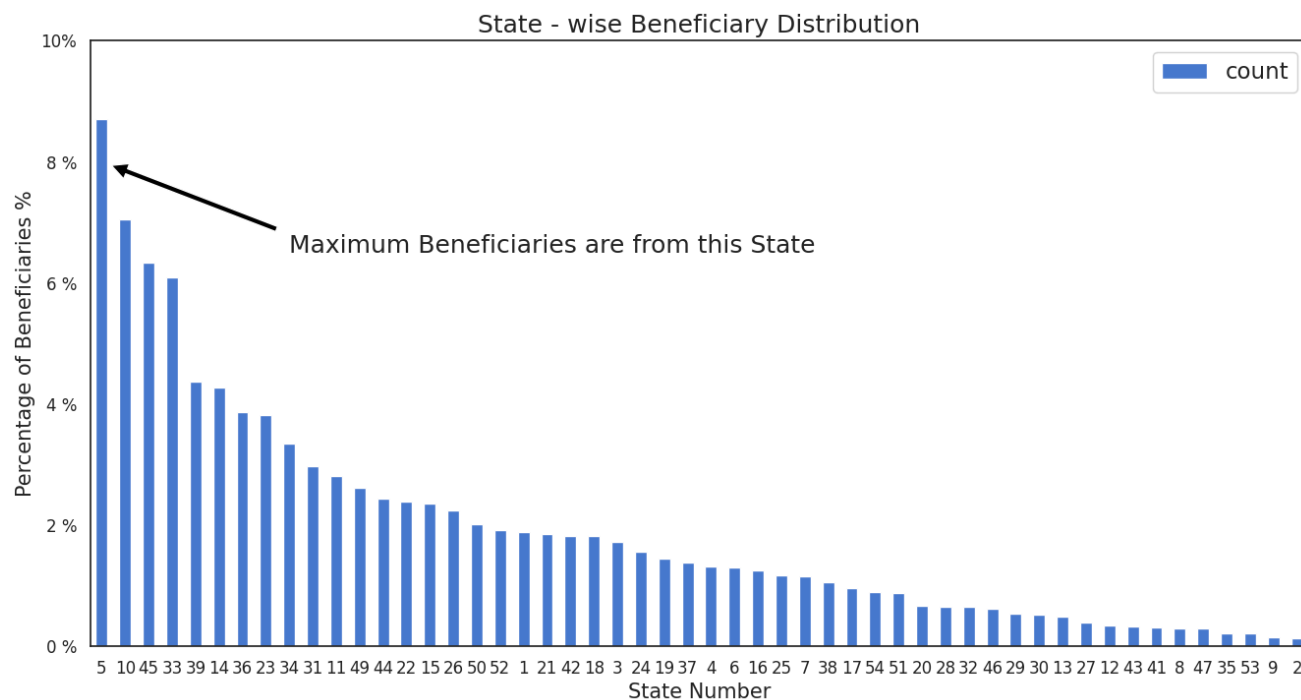
```
plt.annotate('Maximum Beneficiaries are from this State', xy=(0.01,8), xytext=(8, 6.5),
            arrowprops=dict(facecolor='black', shrink=0.05))
```

```
plt.xticks(np.arange(0,12,2), ('0 %', '2 %', '4 %', '6 %', '8 %', '10%'))
```

```
plt.title("State - wise Beneficiary Distribution",fontsize=18)
```

```
plt.xlabel("State Number",fontsize=15)
plt.ylabel("Percentage of Beneficiaries '%',fontsize=15)
plt.show()

plt.savefig('StateWiseBeneficiaryDistribution')
```



<Figure size 1400x800 with 0 Axes>

Above Plot shows top states with their beneficiary percentage distribution. States 5, 10, 45 are top states in terms of beneficiary percentage

Race-wise Percent Distribution of Beneficiaries

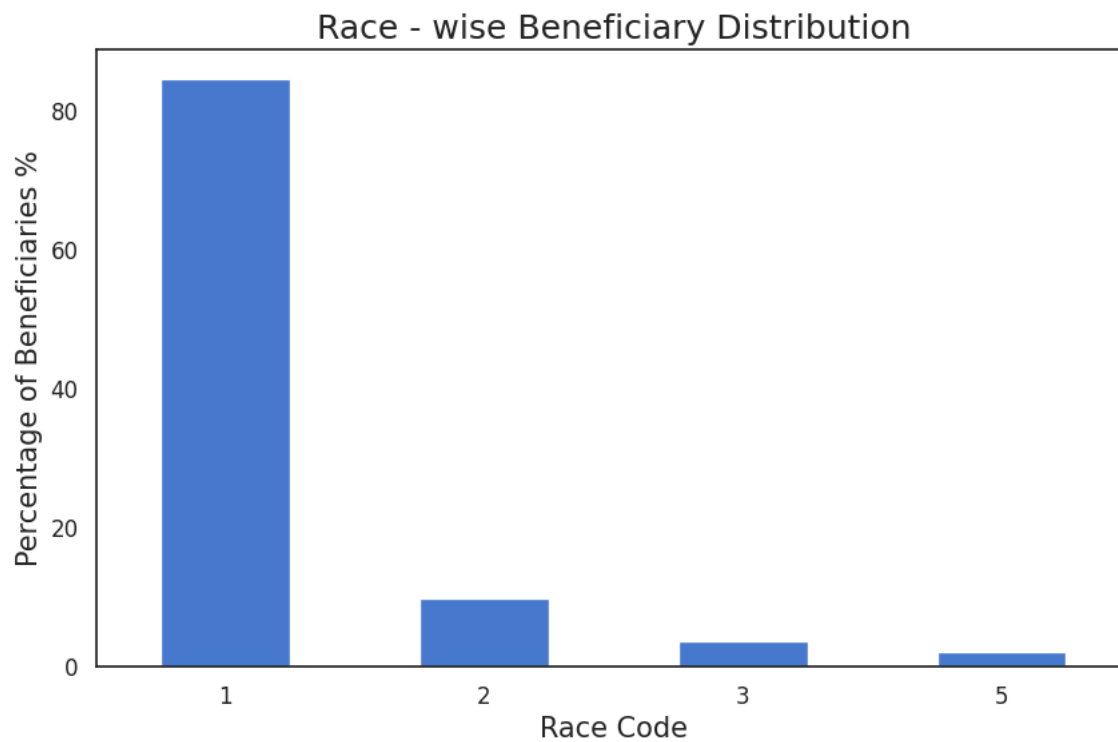
```
#Plotting the frequencies of race-wise beneficiaries
count_Race = pd.value_counts(Train_Beneficiarydata['Race'], sort = True)

#Drawing a barplot
(count_Race*100/len(Train_Beneficiarydata)).plot(kind = 'bar', rot=0,figsize=(10,6),fontsize=12)

#Giving titles and labels to the plot
plt.yticks(np.arange(0,100,20))#, ('0 %', '20 %', '40 %', '60 %', '80 %', '100%'))
plt.title("Race - wise Beneficiary Distribution",fontsize=18)
plt.xlabel("Race Code",fontsize=15)
plt.ylabel("Percentage of Beneficiaries '%',fontsize=15)

plt.show()

plt.savefig('RacewiseBeneficiaryDistribution')
```



<Figure size 1400x800 with 0 Axes>

It seems that more than 80% beneficiaries are of same race which is Race 1 which means Maximum population in the dataset originated from same race. There is no race 4 in the dataset

Top-10 Procedures invloved in Healthcare Fraud

```
## Lets plot countplot for each fraud non fraud categories

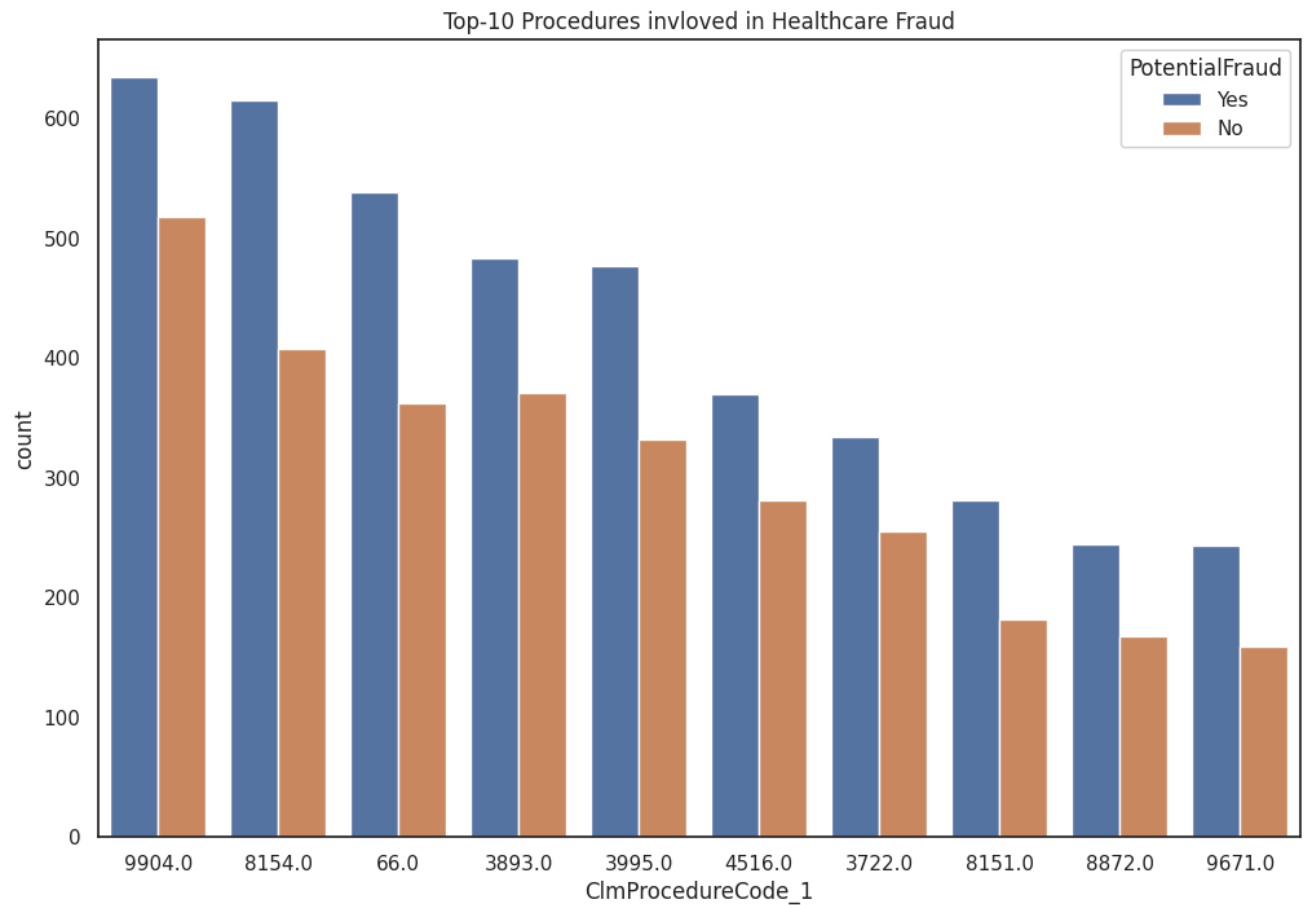
sns.set(rc={'figure.figsize':(12,8)},style='white')

ax=sns.countplot(x='C1mProcedureCode_1',hue='PotentialFraud',data=Train_ProviderWithPatientDetailsdata
,order=Train_ProviderWithPatientDetailsdata.C1mProcedureCode_1.value_counts().iloc[:10].index)

plt.title('Top-10 Procedures invloved in Healthcare Fraud')

plt.show()

plt.savefig('TopProceduresinvlovedinHealthcareFraud')
```



From the above plot, we can say that Procedure 9904, 8154, 66 are top procedures (in terms of money involved). Distribution of fraud and non fraud count shows suspicious transactions involved in them.

Top-10 Diagnosis invloved in Healthcare Fraud

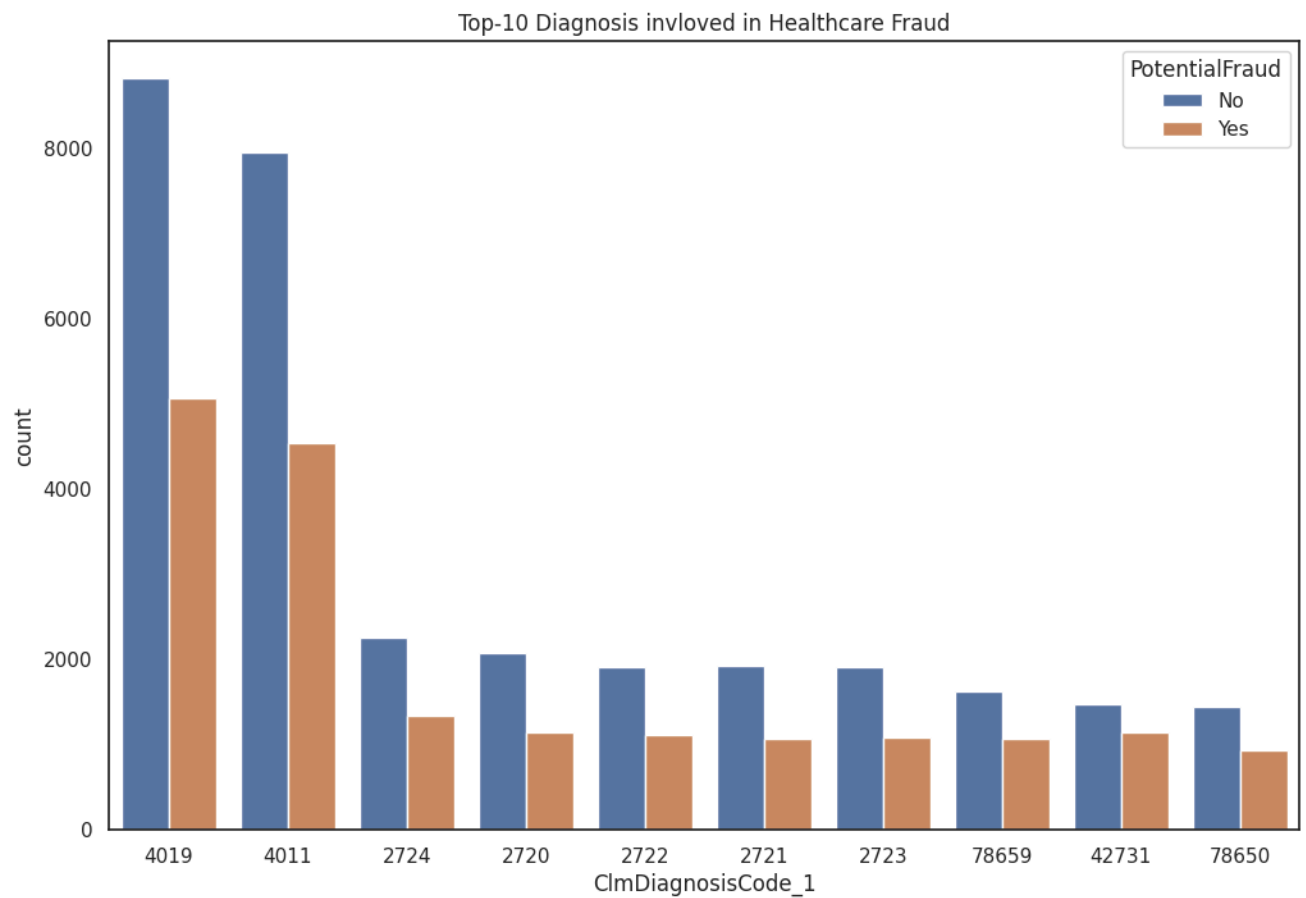
```
## lets plot Top-10 Claim Diagnosis invloved in Healthcare Fraud

sns.set(rc={'figure.figsize':(12,8)},style='white')

sns.countplot(x='CImDiagnosisCode_1',hue='PotentialFraud',data=Train_ProviderWithPatientDetailsdata,
              ,order=Train_ProviderWithPatientDetailsdata.CImDiagnosisCode_1.value_counts().iloc[:10].index)

plt.title('Top-10 Diagnosis invloved in Healthcare Fraud')
plt.show()

plt.savefig('TopDiagnosisInnvlovedinHealthcareFraud')
```

From the above plot, we can say that diagnosis 4019, 4011, 2724 are top diagnosis (in terms of money involved). Distribution of fraud and non fraud counts shows suspicious transactions involved in them.

Top-20 Attending Physicians involved in Healthcare Fraud

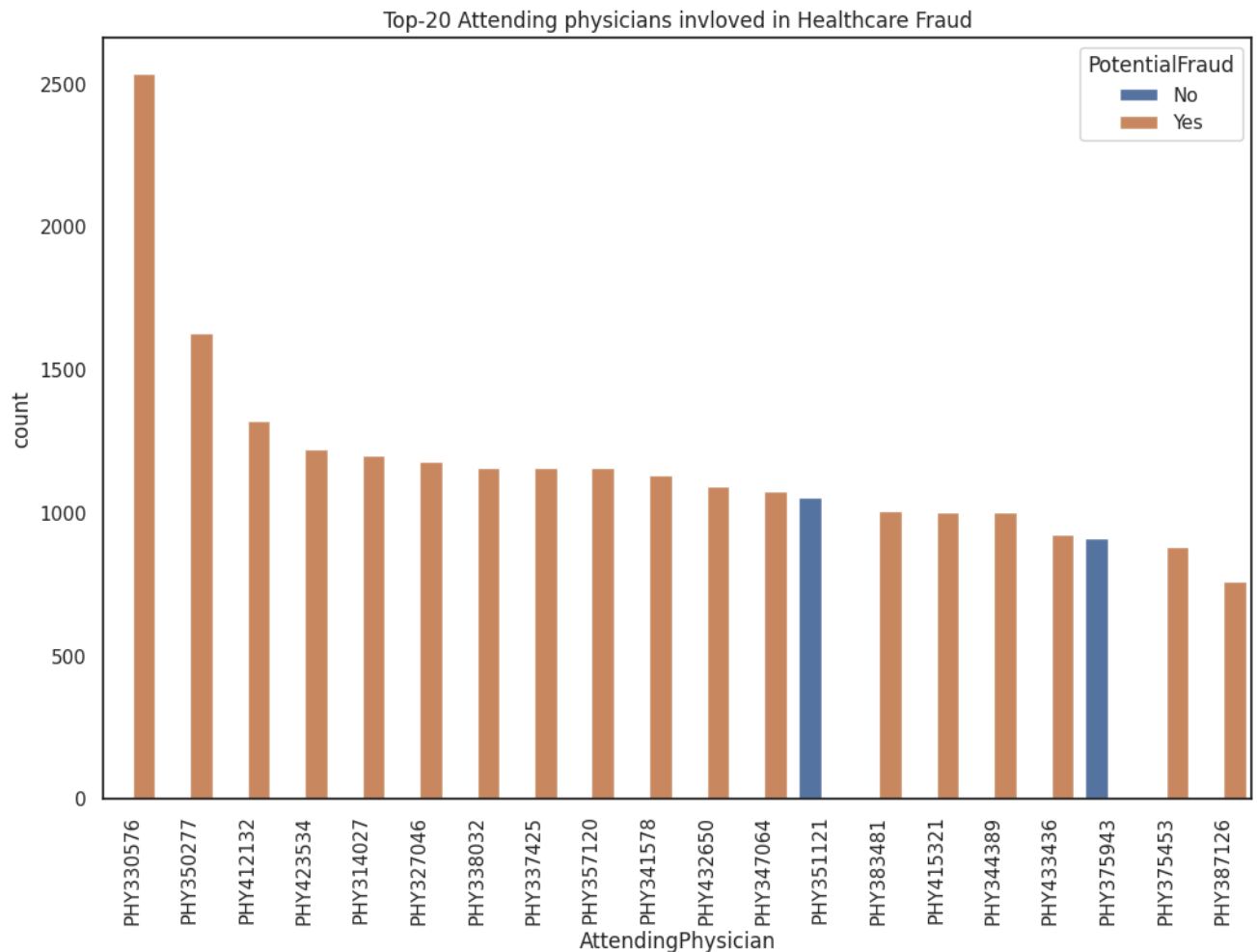
```
### lets plot Top-20 Attending Physicians involved in Healthcare Fraud

sns.set(rc={'figure.figsize':(12,8)},style='white')

ax= sns.countplot(x='AttendingPhysician',hue='PotentialFraud',data=Train_ProviderWithPatientDetailsdata
,order=Train_ProviderWithPatientDetailsdata.AttendingPhysician.value_counts().iloc[:20].index)

plt.title('Top-20 Attending physicians involved in Healthcare Fraud')
plt.xticks(rotation=90)
plt.show()

plt.savefig('TopAttendingphysiciansinvolvedinHealthcareFraud')
```



<Figure size 1200x800 with 0 Axes>

In the above plot, we see count of involvement of attending physicians. And flags the nature of provider where they are working is whether fraud or non fraud.

IPAnnualDeductibleAmt Vs IPAnnualReimbursementAmt Fraud and non Fraud Categories

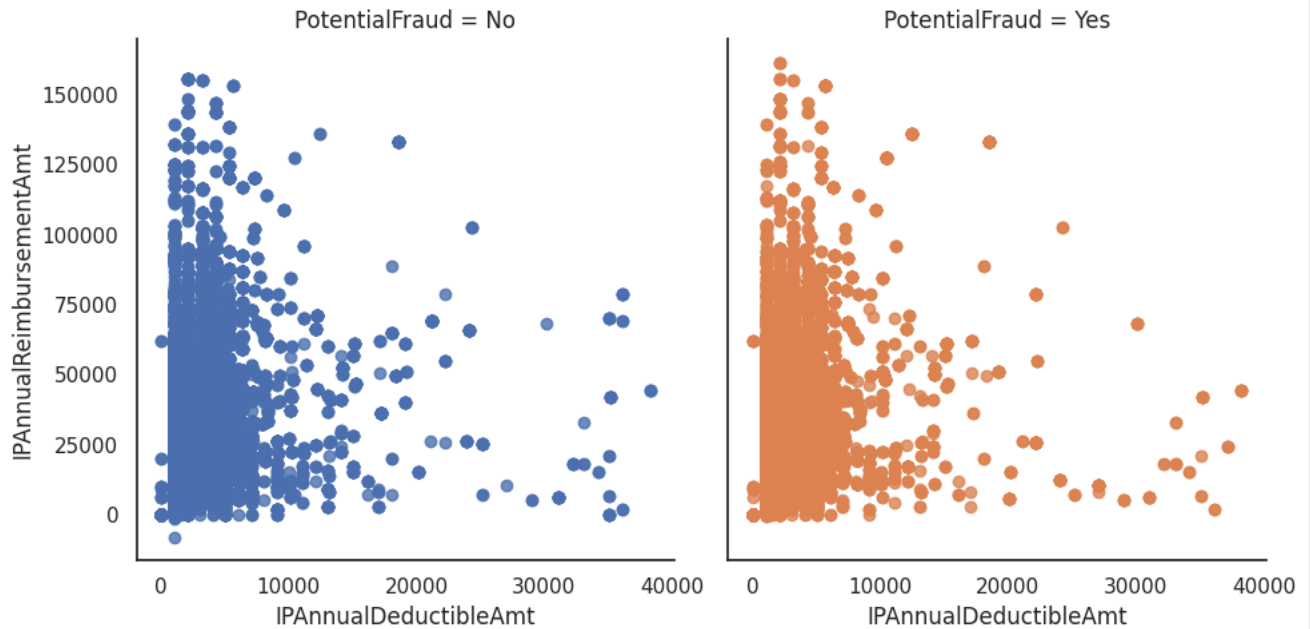
```
## Lets Plot IPAnnualDeductibleAmt and IPAnnualReimbursementAmt in both fraud and non Fraud Categories

sns.set(rc={'figure.figsize':(12,8)},style='white')

sns.lmplot(x='IPAnnualDeductibleAmt',y='IPAnnualReimbursementAmt',hue='PotentialFraud',
           col='PotentialFraud',fit_reg=False,data=Train_ProviderWithPatientDetailsdata)

plt.title('IPAnnualDeductibleAmt and IPAnnualReimbursementAmt in both fraud and non Fraud Categories')

plt.savefig('IPAnnualDeductibleAmtandIPAnnualReimbursementAmtinbothfraudandnonFraud')
```



As we see in the above graph, there is no visible difference in IPAnnualDeductibleAmt and IPAnnualReimbursementAmt.

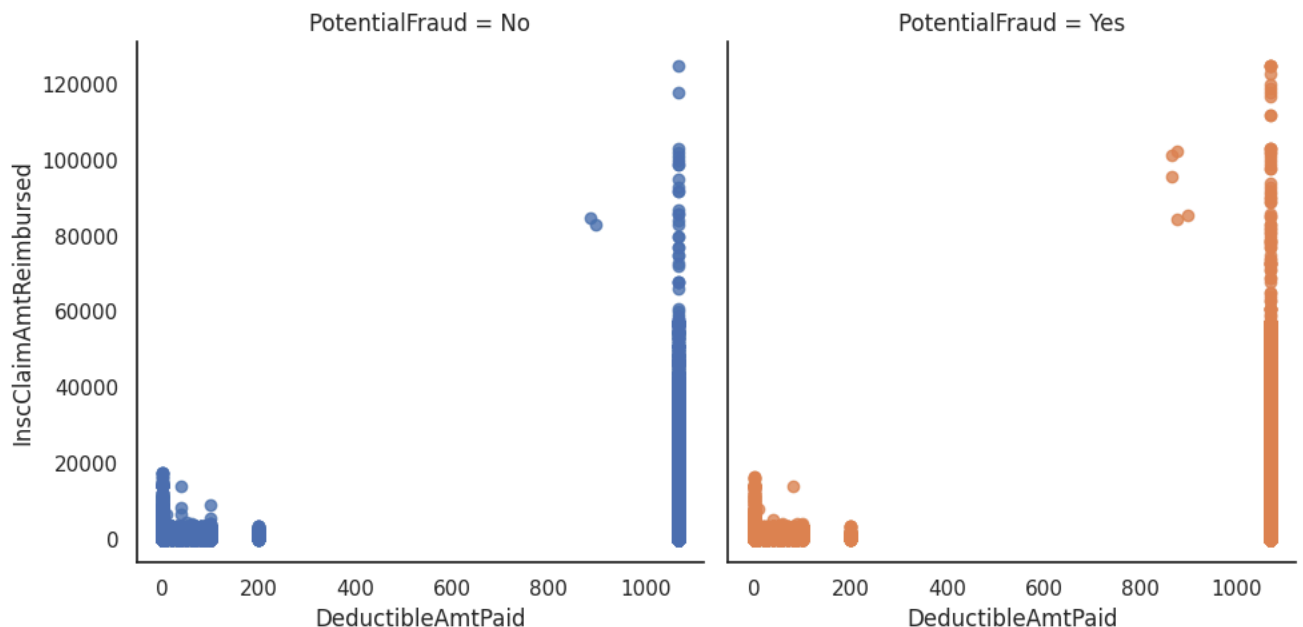
DeductibleAmtPaid Vs InsClaimAmtReimbursed in Fraud and non Fraud Categories

```
## Lets Plot DeductibleAmtPaid and InsClaimAmtReimbursed in both fraud and non Fraud Categories

sns.set(rc={'figure.figsize':(12,8)},style='white')

sns.lmplot(x='DeductibleAmtPaid',y='InsClaimAmtReimbursed',hue='PotentialFraud',
           col='PotentialFraud',fit_reg=False,data=Train_ProviderWithPatientDetailsdata)

plt.savefig('DeductibleAmtPaidandInsClaimAmtReimbursed')
```



We can not differentiate between fraud and non fraud cases based only on DeductibleAmtPaid and InsClaimAmtReimbursed. This lets us derive more features from datasets.

Insurance Claim Amount Reimbursed Vs Age in Fraud and Non Fraud

```

## Let's See Insurance Claim Amount Reimbursed Vs Age
sns.set(rc={'figure.figsize':(12,8)},style='white')

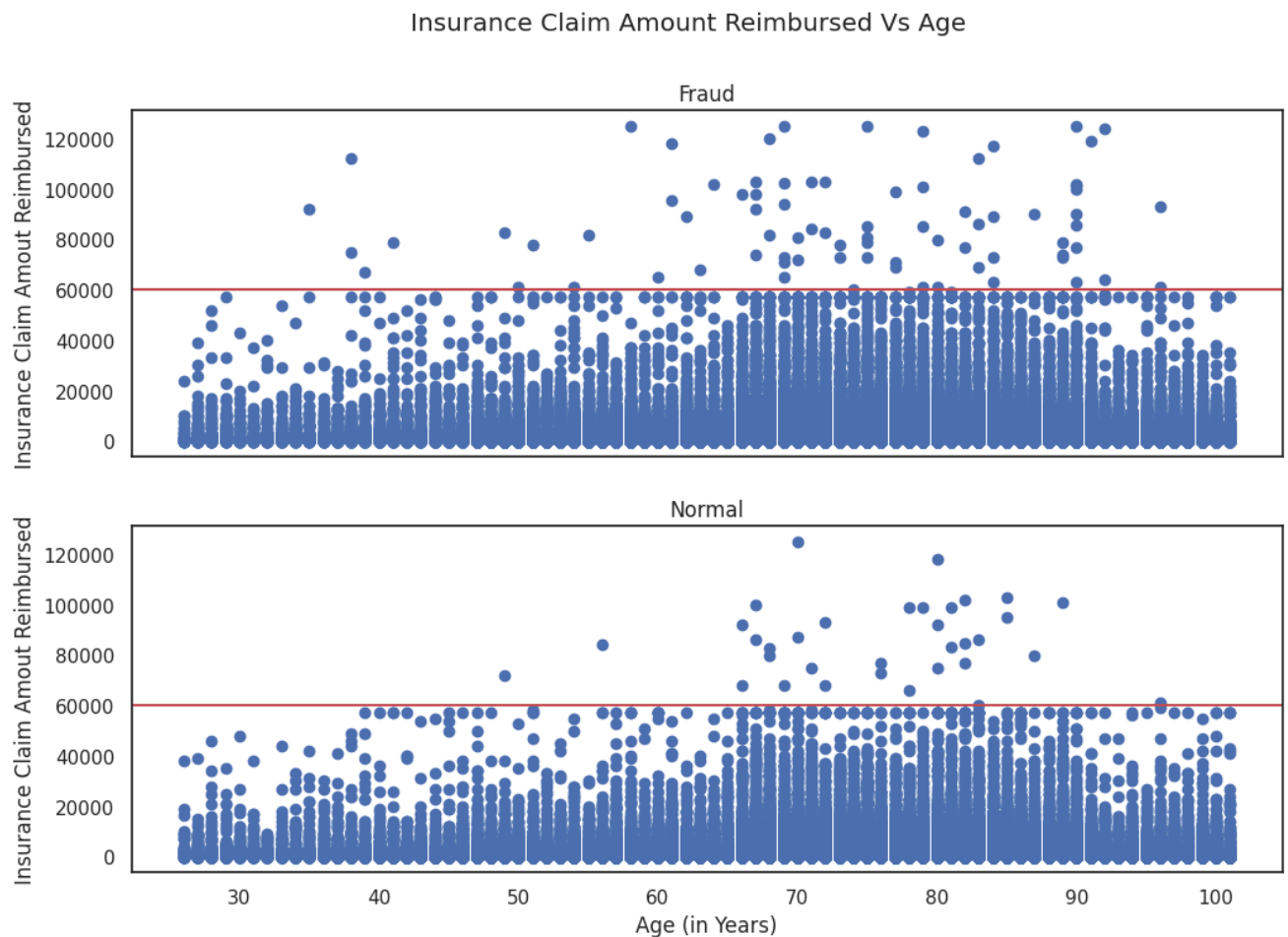
f, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
f.suptitle('Insurance Claim Amount Reimbursed Vs Age')

ax1.scatter(Train_ProviderWithPatientDetailsdata[Train_ProviderWithPatientDetailsdata.PotentialFraud=='Yes'].Age,
            Train_ProviderWithPatientDetailsdata[Train_ProviderWithPatientDetailsdata.PotentialFraud=='Yes'].InscClaimAmtReimbu
ax1.set_title('Fraud')
ax1.axhline(y=60000,c='r')
ax1.set_ylabel('Insurance Claim Amount Reimbursed')

ax2.scatter(Train_ProviderWithPatientDetailsdata[Train_ProviderWithPatientDetailsdata.PotentialFraud=='No'].Age,
            Train_ProviderWithPatientDetailsdata[Train_ProviderWithPatientDetailsdata.PotentialFraud=='No'].InscClaimAmtReimbur
ax2.set_title('Normal')
ax2.axhline(y=60000,c='r')
ax2.set_xlabel('Age (in Years)')
ax2.set_ylabel('Insurance Claim Amount Reimbursed')

plt.show()
f.savefig('AgeVsClaimAmtReimbursed')

```



From the above graph, we see that the occurrence of fraud cases is more frequent in lower age groups (30-70 years) compared to higher age groups (70+ years). Age is one of the important features for differentiating between fraud and non-fraud behaviour.

Feature Engineering Append Train to Test to derive more accurate features.

```
# Lets check the last record of Test_ProviderWithPatientDetailsdata
```

```
Test_ProviderWithPatientDetailsdata.iloc[[135391]]
```

	Provider	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician
135391	PRV57750	BENE99685	CLM541392	2009-08-23	2009-08-23	20	PHY323525	N

```
1 rows × 57 columns
```

```
# Lets check the last record of Train_ProviderWithPatientDetailsdata
```

```
Train_ProviderWithPatientDetailsdata.iloc[[558210]]
```

	Provider	PotentialFraud	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	InscClaimAmtReimbursed	AttendingPhysician
558210	PRV57763	No	BENE97084	CLM662158	2009-11-02	2009-11-02	70	PHY321072

```
1 rows × 58 columns
```

Appending Train data to Test data will help you get good average scores of new features in Test data,as we see not all levels of variables are present in test data compared to train data.So our approach here will be-to append train data to test data ,derive new average features and take only test data to evaluate results.

```
## Lets create a copy of test data first and merge test with train to get better feature averages
```

```
Test_ProviderWithPatientDetailsdata_copy=Test_ProviderWithPatientDetailsdata
```

```
#Lets Check shape of copy
```

```
print('Shape of Test Copy :- ',Test_ProviderWithPatientDetailsdata_copy.shape)
```

```
Shape of Test Copy :- (135392, 57)
```

```
### We will concat two datasets using columns of Test data only as we dont see target column in Test data.
```

```
col_merge=Test_ProviderWithPatientDetailsdata.columns
```

```
## Lets add both test and train datasets
```

```
Test_ProviderWithPatientDetailsdata=pd.concat([Test_ProviderWithPatientDetailsdata,
                                                Train_ProviderWithPatientDetailsdata[col_merge]])
```

```
### Lets verify shape after concatenating
```

```
print("Shape of Test After Concatenation",Test_ProviderWithPatientDetailsdata.shape)
```

```
print("Expected rows after addition",135392+558211)
```

```
Shape of Test After Concatenation (693603, 57)
```

```
Expected rows after addition 693603
```

```
### Lets check record number 135392 as this will be record from train datasets.
```

```
Test_ProviderWithPatientDetailsdata.iloc[[135392]]
```

	Provider	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	0
0	PRV51001	BENE102690	CLM49733	2009-05-04	2009-05-18	42000	PHY383193	PHY378091	

1 rows × 57 columns

****As we verified that our first record is appended to test data correctly,we are all set to derive Average features grouped according to columns of datasets.**

Other than basic explorations and visualizations, we can use certain methods to identify clues of fraud and abuse. One such simple method is 'Grouping based on Similarity'. In this method, we basically group all the records by the ProcedureCodes, DiagnosisCodes,Provider.

For example, if we have a dataset with Procedure codes only for X Procedure, we will then group and check average Amounts involved at each levels of Procedure and analyse the behaviour**

Average Features based on grouping variables. Average features grouped by Provider

```
Train_ProviderWithPatientDetailsdata["PerProviderAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.groupby('Prc
Train_ProviderWithPatientDetailsdata["PerProviderAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby('Provider')
Train_ProviderWithPatientDetailsdata["PerProviderAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby('F
Train_ProviderWithPatientDetailsdata["PerProviderAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby('Prov
Train_ProviderWithPatientDetailsdata["PerProviderAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby('F
Train_ProviderWithPatientDetailsdata["PerProviderAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby('Prov
Train_ProviderWithPatientDetailsdata["PerProviderAvg_Age"]=Train_ProviderWithPatientDetailsdata.groupby('Provider')['Age'].tr
Train_ProviderWithPatientDetailsdata["PerProviderAvg_NoOfMonths_PartACov"]=Train_ProviderWithPatientDetailsdata.groupby('Provic
Train_ProviderWithPatientDetailsdata["PerProviderAvg_NoOfMonths_PartBCov"]=Train_ProviderWithPatientDetailsdata.groupby('Provic
Train_ProviderWithPatientDetailsdata["PerProviderAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Provider')['A

Test_ProviderWithPatientDetailsdata["PerProviderAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby('Provi
Test_ProviderWithPatientDetailsdata["PerProviderAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('Provider')
Test_ProviderWithPatientDetailsdata["PerProviderAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Prc
Test_ProviderWithPatientDetailsdata["PerProviderAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Provic
Test_ProviderWithPatientDetailsdata["PerProviderAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Prc
Test_ProviderWithPatientDetailsdata["PerProviderAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Provic
Test_ProviderWithPatientDetailsdata["PerProviderAvg_Age"]=Test_ProviderWithPatientDetailsdata.groupby('Provider')['Age'].transf
Test_ProviderWithPatientDetailsdata["PerProviderAvg_NoOfMonths_PartACov"]=Test_ProviderWithPatientDetailsdata.groupby('Provider
Test_ProviderWithPatientDetailsdata["PerProviderAvg_NoOfMonths_PartBCov"]=Test_ProviderWithPatientDetailsdata.groupby('Provider
Test_ProviderWithPatientDetailsdata["PerProviderAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Provider')['Adm
```

```
#lets check shape

print('Train',Train_ProviderWithPatientDetailsdata.shape)
Train_ProviderWithPatientDetailsdata.iloc[:,-10:].head(2)

print("Test ",Test_ProviderWithPatientDetailsdata.shape)
Test_ProviderWithPatientDetailsdata.iloc[:,-10:].head(4)
```

Train (558211, 68)
Test (693603, 67)

	PerProviderAvg_InscClaimAmtReimbursed	PerProviderAvg_DeductibleAmtPaid	PerProviderAvg_IPAnnualReimbursementAmt	PerProvider
0	262.390244	1.853659	5180.926829	
1	262.390244	1.853659	5180.926829	
2	262.390244	1.853659	5180.926829	
3	262.390244	1.853659	5180.926829	

Average features grouped by BeneID

```
## Grouping based on BeneID explains amounts involved per beneficiary.Reason to derive this feature is that one beneficiary
## can go to multiple providers and can be involved in fraud cases
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')['De
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')
Train_ProviderWithPatientDetailsdata["PerBeneIDAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('BeneID')['AdmitForDays']

Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')
Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')['De
Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')
Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')
Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')
Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')
Test_ProviderWithPatientDetailsdata["PerBeneIDAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('BeneID')['AdmitForDays']
```

Average features grouped by OtherPhysician

```
### Average features grouped by OtherPhysician.
```

```
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby('Ot
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.grou
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.grou
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby
Train_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('OtherPh

Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('Othe
Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupb
Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupb
Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerOtherPhysicianAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('OtherPhys
```

Average features grouped by OperatingPhysician

```
##Average features grouped by OperatingPhysician
```

```
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.{
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupb
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdat
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Op
```



```

Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerOperatingPhysicianAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Oper

```

Average features grouped by AttendingPhysician

```

### Average features grouped by AttendingPhysician

```

```

Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Att

```

```

Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerAttendingPhysicianAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Atter

```

Average features grouped by DiagnosisGroupCode

```
### Average features grouped by DiagnosisGroupCode
```

```
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Dia
```

```
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.grou
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerDiagnosisGroupCodeAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Diagr
```

Average features grouped by ClmAdmitDiagnosisCode

Average features grouped by ClmAdmitDiagnosisCode

```
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.grou
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsda
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsda
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('

Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.g
Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupb
Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata
Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata
Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmAdmitDiagnosisCodeAvg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Cl
```

Average features grouped by ClmProcedureCode_1

```
### Average features grouped by ClmProcedureCode_1
```

```
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Clm
```

```
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.grou
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmProcedureCode_1Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('ClmPr
```

Average features grouped by ClmProcedureCode_2

```
### Average features grouped by ClmProcedureCode_2
```

```
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_2Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.{
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_2Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.group
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_2Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdat
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_2Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_2Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdat
Train_ProviderWithPatientDetailsdata["PerClmProcedureCode_2Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.gr
```

```

Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_2Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Cln

```

Average features grouped by ClnProcedureCode_3

```

### Average features grouped by ClnProcedureCode_3

```

```

Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.groupby('Cln
Train_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Cln

```

```

Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.groupby('Cln
Test_ProviderWithPatientDetailsdata["PerClnProcedureCode_3Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Cln

```

Average features grouped by ClmDiagnosisCode_1

Average features grouped by ClmDiagnosisCode_1

```
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('ClmDi
```

```
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.grou
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_1Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('ClmDi
```

Average features grouped by ClmDiagnosisCode_2

```
### Average features grouped by ClmDiagnosisCode_2
```

```
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby(
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Clm
```

```
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.grou
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_2Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('ClmDi
```

****Average features grouped by ClmDiagnosisCode_3 ****

Average features grouped by ClmDiagnosisCode_3

```
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.grc
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('Clm
```

```
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.grou
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby('
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.group
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_3Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('ClmDi
```

Average features grouped by ClmDiagnosisCode_4

Average features grouped by ClmDiagnosisCode_4

```
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_InscClaimAmtReimbursed"]=Train_ProviderWithPatientDetailsdata.{
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_DeductibleAmtPaid"]=Train_ProviderWithPatientDetailsdata.groupby
```



```

Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_IPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_IPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_OPAnnualReimbursementAmt"]=Train_ProviderWithPatientDetailsdata.
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_OPAnnualDeductibleAmt"]=Train_ProviderWithPatientDetailsdata.gr
Train_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_AdmitForDays"]=Train_ProviderWithPatientDetailsdata.groupby('C

Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_InscClaimAmtReimbursed"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_DeductibleAmtPaid"]=Test_ProviderWithPatientDetailsdata.groupby(
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_IPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_IPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_OPAnnualReimbursementAmt"]=Test_ProviderWithPatientDetailsdata.
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_OPAnnualDeductibleAmt"]=Test_ProviderWithPatientDetailsdata.gr
Test_ProviderWithPatientDetailsdata["PerClmDiagnosisCode_4Avg_AdmitForDays"]=Test_ProviderWithPatientDetailsdata.groupby('Clm

```

Claims are filed by Provider,so fraud can be organized crime.So we will check ClmCounts filed by Providers and when pairs like Provider +BeneID, Provider+Attending Physician, Provider+ClmAdmitDiagnosisCode, Provider+ClmProcedureCode_1,Provider+ClmDiagnosisCode_1 are together.

Average Feature based on grouping based on combinations of different variables.

```

### Average Feature based on grouping based on combinations of different variables

```

```

Train_ProviderWithPatientDetailsdata["ClmCount_Provider"]=Train_ProviderWithPatientDetailsdata.groupby(['Provider'])['ClaimID
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID"]=Train_ProviderWithPatientDetailsdata.groupby(['Provider','Be
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_AttendingPhysician"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_OtherPhysician"]=Train_ProviderWithPatientDetailsdata.groupby(['Provi
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_OperatingPhysician"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmAdmitDiagnosisCode"]=Train_ProviderWithPatientDetailsdata.groupby([
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmProcedureCode_1"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmProcedureCode_2"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmProcedureCode_3"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmProcedureCode_4"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmProcedureCode_5"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr
Train_ProviderWithPatientDetailsdata["ClmCount_Provider_ClmDiagnosisCode_1"]=Train_ProviderWithPatientDetailsdata.groupby(['Pr

```

```
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_AttendingPhysician"]=Test_ProviderWithPatientDetailsdata.groupby(["ClmCount_Provider_BeneID_OtherPhysician"])
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_OtherPhysician"]]=Test_ProviderWithPatientDetailsdata.groupby(["ClmCount_Provider_BeneID_AttendingPhysician_ClmProcedureCode_1"])=Test_ProviderWithPatient
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_AttendingPhysician_ClmDiagnosisCode_1"]]=Test_ProviderWithPatient
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_OperatingPhysician"]]=Test_ProviderWithPatientDetailsdata.groupby
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_ClmProcedureCode_1"]]=Test_ProviderWithPatientDetailsdata.groupby
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_ClmDiagnosisCode_1"]]=Test_ProviderWithPatientDetailsdata.groupby
Test_ProviderWithPatientDetailsdata["ClmCount_Provider_BeneID_ClmDiagnosisCode_1_ClmProcedureCode_1"]]=Test_ProviderWithPatient
```



```
## Lets Check Shape after creating new features.
```

```
print('Train_ProviderWithPatientDetailsdata shape-',Train_ProviderWithPatientDetailsdata.shape)
print('Test_ProviderWithPatientDetailsdata shape-',Test_ProviderWithPatientDetailsdata.shape)
```

```
Train_ProviderWithPatientDetailsdata shape- (558211, 188)
Test_ProviderWithPatientDetailsdata shape- (693603, 187)
```

```
## Lets Check unique values of ICD Diagnosis Codes
```

```
diagnosiscode_2chars=Train_ProviderWithPatientDetailsdata['C1mDiagnosisCode_1'].astype(str).str[0:2]
```

```
diagnosiscode_2chars.unique()
```

```
array(['42', 'V7', '59', '24', '78', '41', '56', '79', '04', '35', '85',
       '51', 'V4', '29', '03', 'na', '71', 'V1', '27', '60', '50', '57',
       '28', '91', '55', 'V5', '00', '15', '99', '82', '72', '19', '40',
       '58', '73', '70', '25', '46', '49', '44', '45', '18', '16', '68',
       '61', '47', '20', '33', '31', '88', '48', '53', '43', '36', 'V6',
       '83', 'E8', '02', '32', '17', 'V0', '34', '62', 'V8', '11', '30',
       '38', '39', '80', '93', '52', '84', '89', '23', '94', '54', 'V2',
       '26', '37', '87', '21', '05', '69', '81', '90', '12', '92', '63',
       '64', '95', '22', '07', '06', '13', '74', '14', '98', '97', '08',
       '66', '75', 'E9', '09', '96', '10', '77', '86', '65', '67', '01',
       '76'], dtype=object)
```

```
x=diagnosiscode_2chars.sort_values(ascending=True)
```

```
x=diagnosiscode_2chars.sort_values(ascending=True)
x.unique()
```

```
array(['00', '01', '02', '03', '04', '05', '06', '07', '08', '09', '10',
       '11', '12', '13', '14', '15', '16', '17', '18', '19', '20', '21',
       '22', '23', '24', '25', '26', '27', '28', '29', '30', '31', '32',
       '33', '34', '35', '36', '37', '38', '39', '40', '41', '42', '43',
       '44', '45', '46', '47', '48', '49', '50', '51', '52', '53', '54',
       '55', '56', '57', '58', '59', '60', '61', '62', '63', '64', '65',
       '66', '67', '68', '69', '70', '71', '72', '73', '74', '75', '76',
       '77', '78', '79', '80', '81', '82', '83', '84', '85', '86', '87',
       '88', '89', '90', '91', '92', '93', '94', '95', '96', '97', '98',
       '99', 'E8', 'E9', 'V0', 'V1', 'V2', 'V4', 'V5', 'V6', 'V7', 'V8',
       'na'], dtype=object)
```

Above Data Shows that if we take only first 2 characters of diagnosis code for the purpose of grouping them ,we might end up creating large sparse matrix ,as each 'code' column will generate 120+ dummy columns.This will increase computational time and loose explicability.

Data Preprocessing

Impute Numeric columns with 0's.

```
##### Lets impute numeric columns with 0

cols1 = Train_ProviderWithPatientDetailsdata.select_dtypes([np.number]).columns
cols2 = Train_ProviderWithPatientDetailsdata.select_dtypes(exclude = [np.number]).columns

Train_ProviderWithPatientDetailsdata[cols1] = Train_ProviderWithPatientDetailsdata[cols1].fillna(value=0)
Test_ProviderWithPatientDetailsdata[cols1]=Test_ProviderWithPatientDetailsdata[cols1].fillna(value=0)
print('Test_ProviderWithPatientDetailsdata shape:',Test_ProviderWithPatientDetailsdata.shape)
```

```
Test_ProviderWithPatientDetailsdata shape: (693603, 187)
```

Feature Selection

```
# Lets remove unnecessary columns ,as we grouped based on these columns and derived maximum information from them.

cols=Train_ProviderWithPatientDetailsdata.columns
cols[:58]

remove_these_columns=['BeneID', 'ClaimID', 'ClaimStartDt','ClaimEndDt','AttendingPhysician',
    'OperatingPhysician', 'OtherPhysician', 'ClmDiagnosisCode_1',
    'ClmDiagnosisCode_2', 'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4',
    'ClmDiagnosisCode_5', 'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7',
    'ClmDiagnosisCode_8', 'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10',
    'ClmProcedureCode_1', 'ClmProcedureCode_2', 'ClmProcedureCode_3',
    'ClmProcedureCode_4', 'ClmProcedureCode_5', 'ClmProcedureCode_6',
    'ClmAdmitDiagnosisCode', 'AdmissionDt',
    'DischargeDt', 'DiagnosisGroupCode','DOB', 'DOD',
    'State', 'County']

Train_category_removed=Train_ProviderWithPatientDetailsdata.drop(axis=1,columns=remove_these_columns)
Test_category_removed=Test_ProviderWithPatientDetailsdata.drop(axis=1,columns=remove_these_columns)
```

```
## Lets Check shape and missing values
```

```
print('Train Shape :',Train_category_removed.shape)
print('Test Shape : ',Test_category_removed.shape)
print("Train Missing Values",Train_category_removed.isnull().sum().sum())
print("Test Missing Values",Test_category_removed.isnull().sum().sum())
```

```
Train Shape : (558211, 157)
Test Shape : (693603, 156)
Train Missing Values 0
Test Missing Values 0
```

Type Conversion

```
## Lets Convert types of gender and race to categorical.
```

```
Train_category_removed.Gender=Train_category_removed.Gender.astype('category')
Test_category_removed.Gender=Test_category_removed.Gender.astype('category')

Train_category_removed.Race=Train_category_removed.Race.astype('category')
Test_category_removed.Race=Test_category_removed.Race.astype('category')
```

Dummification

```
# Lets create dummies for categorrical columns.

Train_category_removed=pd.get_dummies(Train_category_removed,columns=[ 'Gender', 'Race'],drop_first=True)
Test_category_removed=pd.get_dummies(Test_category_removed,columns=[ 'Gender', 'Race'],drop_first=True)
```

```
Train_category_removed.head().T
Test_category_removed.iloc[135391:135393]
```

	Provider	InscClaimAmtReimbursed	DeductibleAmtPaid	AdmitForDays	RenalDiseaseIndicator	NoOfMonths_PartACov	NoOfMonti
135391	PRV57750	20	0.0	0.0	0	12	
0	PRV51001	42000	1068.0	15.0	1	12	

2 rows × 158 columns

Convert Target values to 1 and 0,wher '1' means Yes and '0' means No

```
Train_category_removed.PotentialFraud.replace(['Yes','No'],[ '1','0'],inplace=True)
Train_category_removed.head()
Train_category_removed.PotentialFraud=Train_category_removed.PotentialFraud.astype('int64')
Train_category_removed.PotentialFraud.dtypes
Train_category_removed.PotentialFraud.min()
```

0

```
Train_category_removed.PotentialFraud.max()
```

1

Select only Test related data from merged data

```
print('Test Shape before removing',Test_category_removed.shape)
```

Test Shape before removing (693603, 158)

```
Test_category_removed=Test_category_removed.iloc[:135392]    ##Remove train data from appended test data
```

```
Test_category_removed.tail()    # Check last 5 records.
```

	Provider	InscClaimAmtReimbursed	DeductibleAmtPaid	AdmitForDays	RenalDiseaseIndicator	NoOfMonths_PartACov	NoOfMonti
135387	PRV57750	50	0.0	0.0	0	12	
135388	PRV57750	0	0.0	0.0	0	12	
135389	PRV57750	100	0.0	0.0	0	12	
135390	PRV57750	20	0.0	0.0	0	12	
135391	PRV57750	20	0.0	0.0	0	12	

5 rows × 158 columns

```
print('Test Shape before removing',Test_category_removed.shape)
```

Test Shape before removing (135392, 158)

Data Aggregation to the Providers level

```
### Lets aggregate claims data to unique providers.
```

```
# Select only numeric columns for summation, excluding the grouping keys
```

```
numeric_cols_train = Train_category_removed.select_dtypes(include=np.number).columns.drop('PotentialFraud').tolist()
Train_category_removed_groupedbyProv_PF=Train_category_removed.groupby(['Provider', 'PotentialFraud'],as_index=False)[numeric_cols_train].agg('sum')
```

```
numeric_cols_test = Test_category_removed.select_dtypes(include=np.number).columns.tolist()
```

```
Test_category_removed_groupedbyProv_PF=Test_category_removed.groupby(['Provider'],as_index=False)[numeric_cols_test].agg('sum')
```

```
print('Providers in Train:',Train_category_removed_groupedbyProv_PF.shape)
```

```
print('Providers in Test :',Test_category_removed_groupedbyProv_PF.shape)
```

```
Providers in Train: (5410, 154)
```

```
Providers in Test : (1353, 153)
```

```
Train_category_removed_groupedbyProv_PF.head(3)
```

```
Test_category_removed_groupedbyProv_PF.head(3)
```

	Provider	InscClaimAmtReimbursed	DeductibleAmtPaid	AdmitForDays	NoOfMonths_PartACov	NoOfMonths_PartBCov	ChronicCond_Alz
0	PRV51002	53790	380.0	0.0	2424	2460	
1	PRV51006	30720	0.0	0.0	1224	1224	
2	PRV51009	27230	1238.0	8.0	468	468	

3 rows × 153 columns

Train Validation split

```
print('Train shape:',Train_category_removed_groupedbyProv_PF.shape)
```

```
Train shape: (5410, 154)
```

```
Train_category_removed_groupedbyProv_PF.head(3)
```

	Provider	PotentialFraud	InscClaimAmtReimbursed	DeductibleAmtPaid	AdmitForDays	NoOfMonths_PartACov	NoOfMonths_PartBCov
0	PRV51001	0	104640	5340.0	30.0	300	300
1	PRV51003	1	605670	66286.0	382.0	1560	1567
2	PRV51004	0	52170	310.0	0.0	1768	1782

3 rows × 154 columns

```
## Lets Separate out Target and providers from independent variables.Create Target column y.
```

```
X=Train_category_removed_groupedbyProv_PF.drop(axis=1,columns=['Provider','PotentialFraud'])
```

```
y=Train_category_removed_groupedbyProv_PF['PotentialFraud']
```

Standardization

```
## Lets apply StandardScaler and transform values to its z form,where 99.7% values range between -3 to 3.
```

```
sc=StandardScaler() # MinMaxScaler
```

```
sc.fit(X)
```

```
X_std=sc.transform(X)
```

```
X_teststd=sc.transform(Test_category_removed_groupedbyProv_PF.iloc[:,1:]) #Apply Standard Scaler to unseen data as well.
```

```
#print(X_std[0:3,:])
```

```
#print(X_teststd[0:3,:])
```

```
print('X Shape:',X_std.shape)
```

```
X Shape: (5410, 152)
```

Lets Split data in train and validation

```
## 'stratify=y' will make sure equal distribution of yes:no in both train and validation
```

```
X_train,X_val,y_train,y_val = train_test_split(X_std,y,test_size=0.3,random_state=101,stratify=y,shuffle=True)
```

```
print('X_train :',X_train.shape)
print('y_train :',y_train.shape)
```

```
print('X_val :',X_val.shape)
print('y_val :',y_val.shape)
```

```
X_train : (3787, 152)
y_train : (3787,)
X_val : (1623, 152)
y_val : (1623,)
```

```
print('X_teststd',X_teststd.shape)
```

```
X_teststd (1353, 152)
```

Model Building

Logistic regression**

```
from sklearn.linear_model import LogisticRegressionCV
```

```
log = LogisticRegressionCV(cv=10,class_weight='balanced',random_state=123)
```

```
# The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies
# in the input data as ``n_samples / (n_classes * np.bincount(y))``.
```

```
log.fit(X_train,y_train)
```

```
LogisticRegressionCV
LogisticRegressionCV(class_weight='balanced', cv=10, random_state=123)
```

```
## Lets predict probability of 1 and 0 for X_train and X_val
```

```
log_train_pred_probability=log.predict_proba(X_train)
log_val_pred_probability=log.predict_proba(X_val)
```

Logistic Regression Model Prediction for Train and Validation

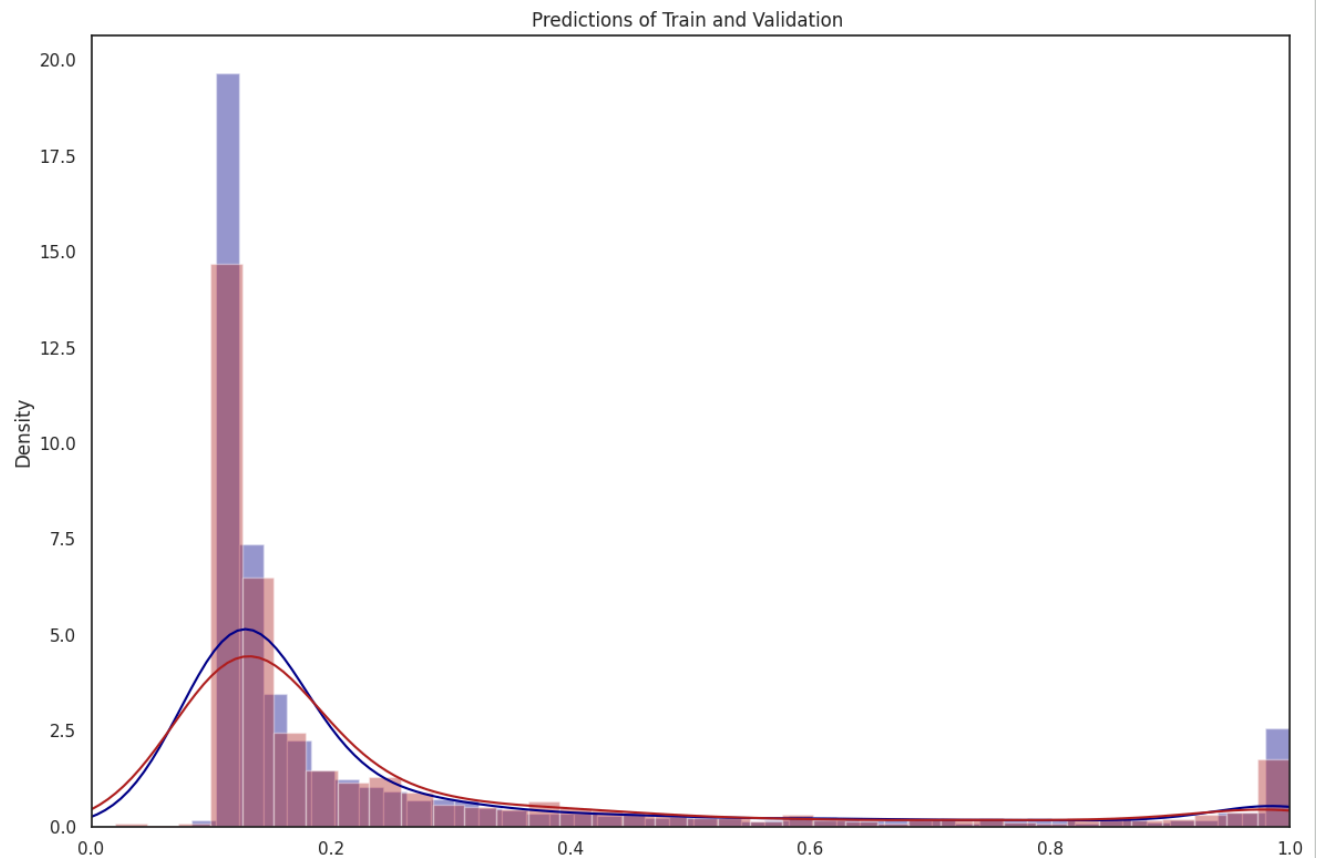
```
# Lets compare model prediction performance on train and Validation both.
```

```
fig = plt.figure(figsize=(12,8))
```

```
sns.distplot(log.predict_proba(X_train)[:,:1],color='darkblue')
sns.distplot(log.predict_proba(X_val)[:,:1],color='firebrick')
plt.title('Predictions of Train and Validation ')
plt.xlim([0, 1])
```

```
plt.tight_layout()
```

```
plt.show()
```

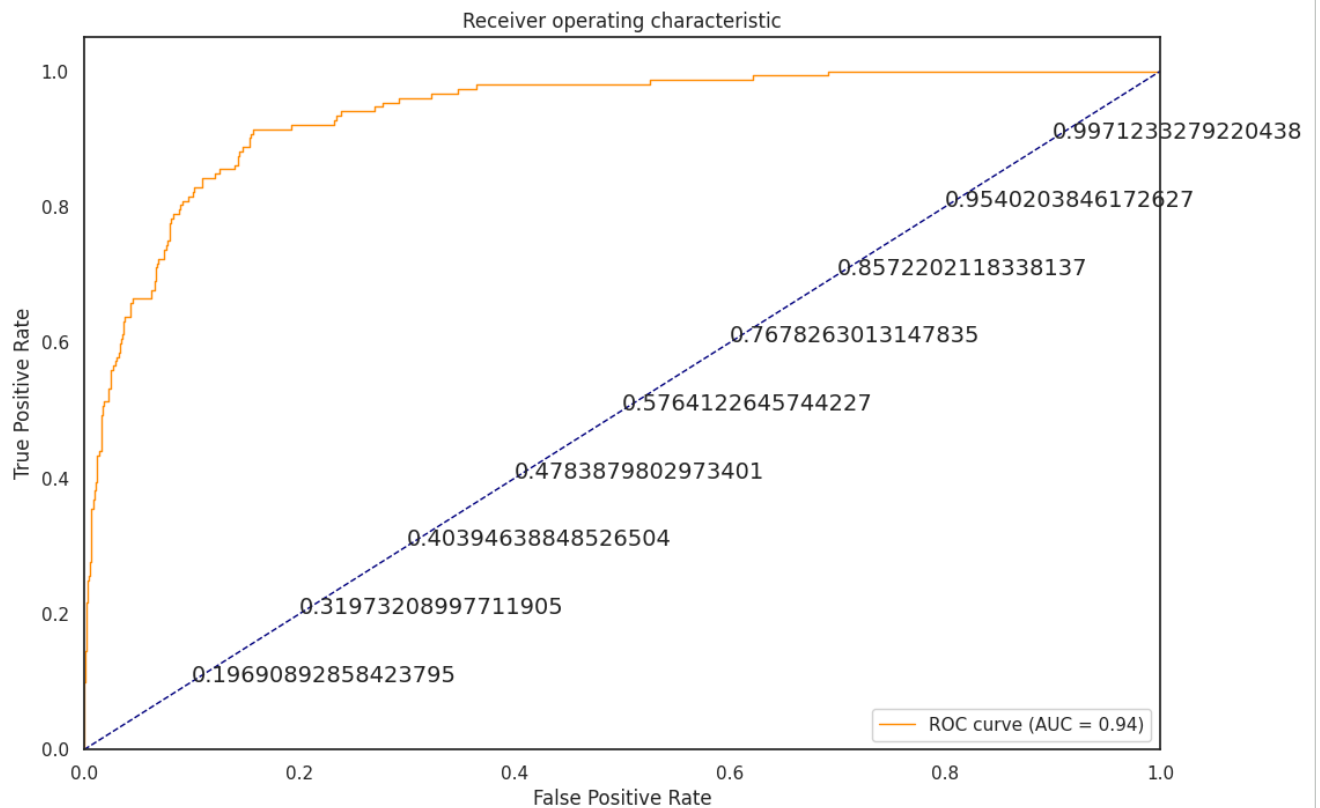
Logistic regression : ROC Curve

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve
fpr, tpr, thresholds = roc_curve(y_val, log.predict_proba(X_val)[: ,1])
#log_val_pred_probability[:,1]
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=1, label='ROC curve (AUC = %0.2f)' %
roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=1, linestyle='--')

for label in range(1,10,1):
    plt.text((10-label)/10,(10-label)/10,thresholds[label*15],fontdict={'size':
14})

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```



```
fpr, tpr, thresholds = roc_curve(y_val, log.predict_proba(X_val)[: ,1], pos_label=1)    #log_val_pred_probability[: ,1])
roc_auc = auc(fpr, tpr)
print("Area under the ROC curve : %f" % roc_auc)
```

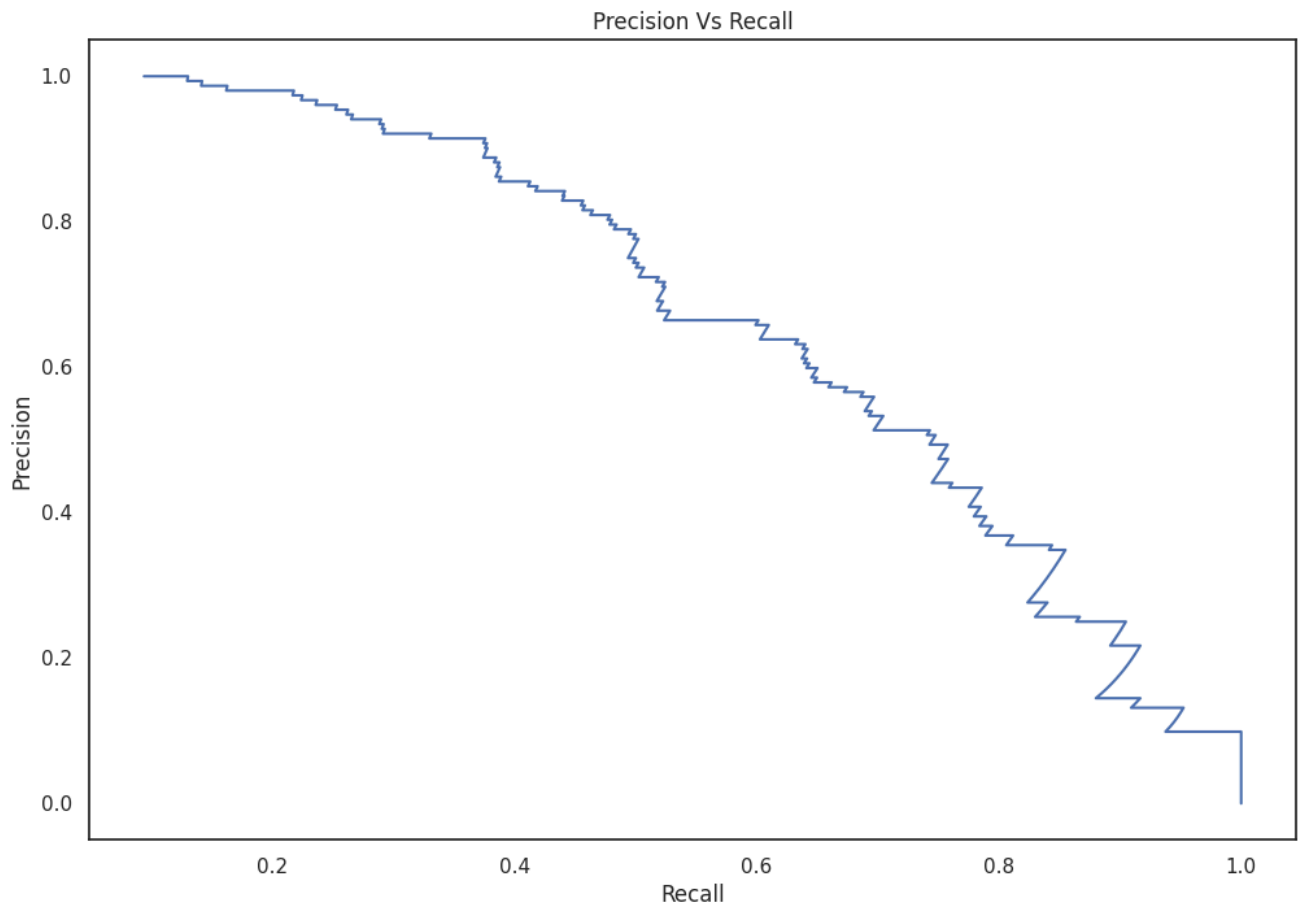
Area under the ROC curve : 0.937051

Logistic Regression : Precision Vs Recall Curve¶

```
## Lets Check precision and recall
precision, recall, _ = precision_recall_curve(y_val, log.predict_proba(X_val)[: ,
1])

plt.plot(precision, recall)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision Vs Recall')
```

Text(0.5, 1.0, 'Precision Vs Recall')



Logistic Regression : TPR Vs FPR

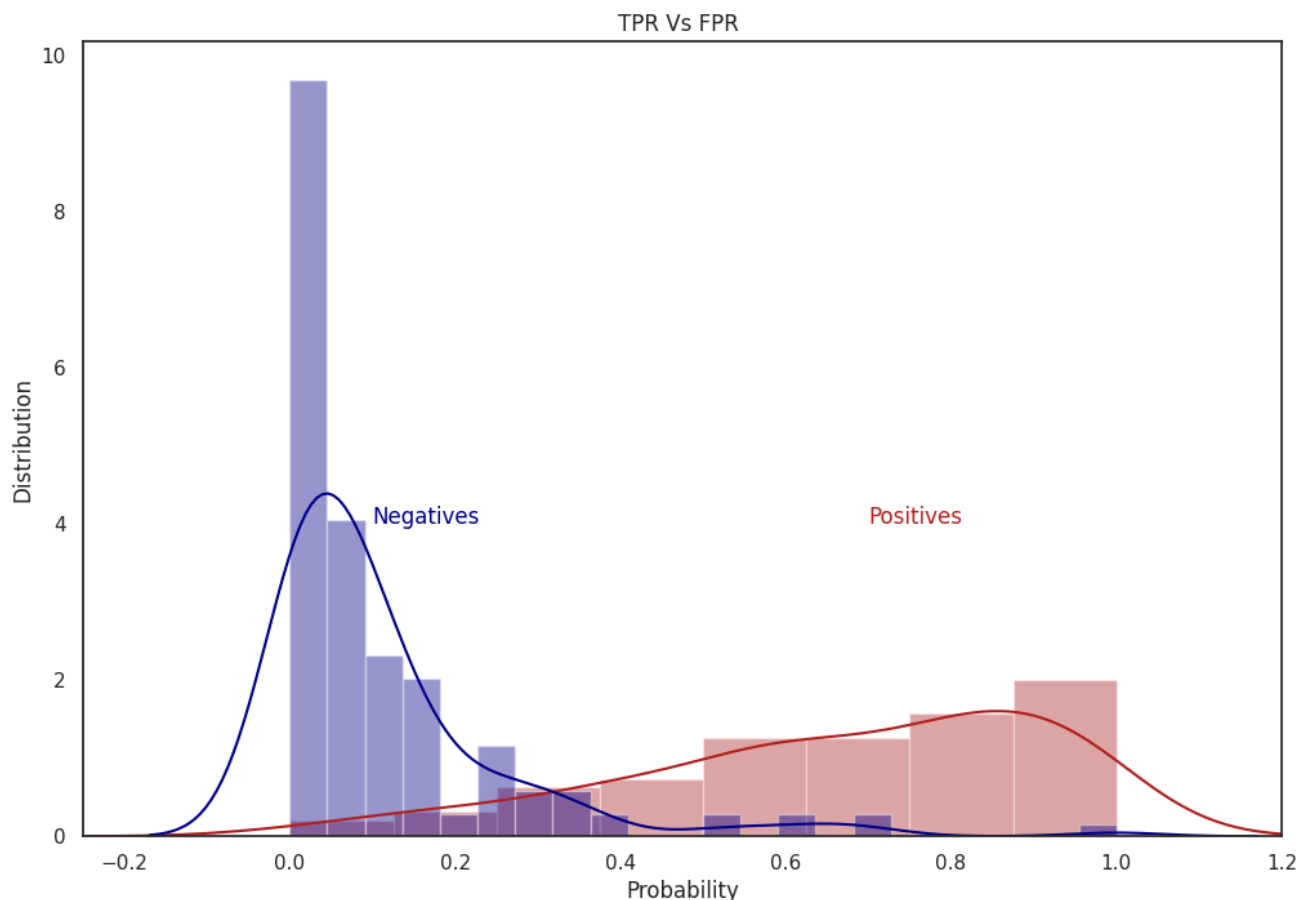
```
## Lets Check Tpr vs fpr distribution.

fig = plt.figure(figsize=(12,8))

sns.distplot(tpr,color='firebrick')

sns.distplot(fpr,color='darkblue')
plt.title('TPR Vs FPR ')
plt.xlim([-0.25, 1.2])

plt.text(0.1,4,'Negatives',color='darkblue')
plt.text(0.7,4,'Positives',color='firebrick')
plt.xlabel('Probability')
plt.ylabel('Distribution')
plt.show()
```



Probability Thresholding

****Thresholding Probabilities:**

If the algorithm returns probabilities (or some other score), thresholding can be applied after a model has been built. Essentially you change the classification threshold from 50-50 to an appropriate trade-off level. This typically can be optimized by generated a curve of the evaluation metric (e.g. F-measure). The limitation here is that you are making absolute trade-offs. Any modification in the cutoff will in turn decrease the accuracy of predicting the other class.**

```
## Lets Set probability Threshold to 0.60
```

```
log_train_pred_60=(log.predict_proba(X_train)[: ,1]>0.60).astype(bool)
log_val_pred_60=(log.predict_proba(X_val)[: ,1]>0.60).astype(bool) # set threshold as 0.60
```

Logistic Regression : Evaluation

```
#Confusion matrix, Accuracy, sensitivity and specificity

from sklearn.metrics import confusion_matrix,accuracy_score,cohen_kappa_score,
roc_auc_score,f1_score,auc

cm0 = confusion_matrix(y_train, log_train_pred_60,labels=[1,0])
print('Confusion Matrix Train : \n', cm0)

cm1 = confusion_matrix(y_val, log_val_pred_60,labels=[1,0])
print('Confusion Matrix Val: \n', cm1)

total0=sum(sum(cm0))
total1=sum(sum(cm1))
#####from confusion matrix calculate accuracy
accuracy0=(cm0[0,0]+cm0[1,1])/total0
print ('Accuracy Train: ', accuracy0)

accuracy1=(cm1[0,0]+cm1[1,1])/total1
```

```
print ('Accuracy Val: ', accuracy1)

sensitivity0 = cm0[0,0]/(cm0[0,0]+cm0[0,1])
print('Sensitivity Train : ', sensitivity0 )

sensitivity1 = cm1[0,0]/(cm1[0,0]+cm1[0,1])
print('Sensitivity Val: ', sensitivity1 )

specificity0 = cm0[1,1]/(cm0[1,0]+cm0[1,1])
print('Specificity Train: ', specificity0)

specificity1 = cm1[1,1]/(cm1[1,0]+cm1[1,1])
print('Specificity Val: ', specificity1)

KappaValue=cohen_kappa_score(y_val, log_val_pred_60)
print("Kappa Value :",KappaValue)
AUC=roc_auc_score(y_val, log_val_pred_60)

print("AUC          :",AUC)

print("F1-Score Train  :",f1_score(y_train, log_train_pred_60))

print("F1-Score Val   :",f1_score(y_val, log_val_pred_60))
```

Confusion Matrix Train :

[[	269	85]
[	209	3224]]

Confusion Matrix Val:

[[	102	50]
[	92	1379]]

Accuracy Train: 0.922365988909427
Accuracy Val: 0.9125077017868145
Sensitivity Train : 0.7598870056497176
Sensitivity Val: 0.6710526315789473
Specificity Train: 0.9391203029420332
Specificity Val: 0.9374575118966689
Kappa Value : 0.5414360243701526
AUC : 0.8042550717378081
F1-Score Train : 0.6466346153846154
F1-Score Val : 0.5895953757225434

Confusion Matrix Confusion Matrix Train : [[270 84] [210 3223]] Confusion Matrix Val: [[103 49] [93 1378]] Accuracy Train: 0.922365988909427 Accuracy Val: 0.9125077017868145 Sensitivity Train : 0.7627118644067796 Sensitivity Val: 0.6776315789473685 Specificity Train: 0.9388290125254879 Specificity Val: 0.9367777022433719 Kappa Value : 0.5438304105142315 AUC : 0.8072046405953702 F1-Score Train : 0.6474820143884892 F1-Score Val : 0.5919540229885056

```
## Lets predict on Test data

log_test_pred_60 = (log.predict_proba(X_teststd)[: ,1]>0.60).astype(bool)
log_test_pred=pd.DataFrame(log_test_pred_60)
log_test_pred.head(2)
```

0	
0	False
1	False

Next steps: [Generate code with log_test_pred](#) [New interactive sheet](#)

```
# Lets Replace 1 with Yes and 0 With No
Replacement = {1:'Yes',0:'No'}

Labels=log_test_pred[0].apply(lambda x : Replacement[x])
Labels.value_counts()      #Check count of Predictions
```

```

      count
0
No      1183
Yes      170

dtype: int64

```

```
## Submission file
```

```

submission_log=pd.DataFrame({"Provider":Test_category_removed_groupedbyProv_PF.Provider})
submission_log['PotentialFraud']=Labels
submission_log.shape

```

```
(1353, 2)
```

```
print('Submission Shape:',submission_log.shape)
```

```
Submission Shape: (1353, 2)
```

```
#write Submission file
```

```
submission_log.to_csv("Submission_Logistic_Regression_F1_60_Threshold_60Prct.csv",index=False)
```

```
submission_log.PotentialFraud.value_counts()
```

```

      count
PotentialFraud
No      1183
Yes      170

dtype: int64

```

Random Forest

```

## Lets Apply Random Forest
from sklearn.ensemble import RandomForestClassifier

```

```
rfc = RandomForestClassifier(n_estimators=500,class_weight='balanced',random_state=123,max_depth=4) # We will set max_depth =
```

```
rfc.fit(X_train,y_train) #fit the model
```

```

RandomForestClassifier
RandomForestClassifier(class_weight='balanced', max_depth=4, n_estimators=500,
random_state=123)

```

Random Forest : ROC Curve

```

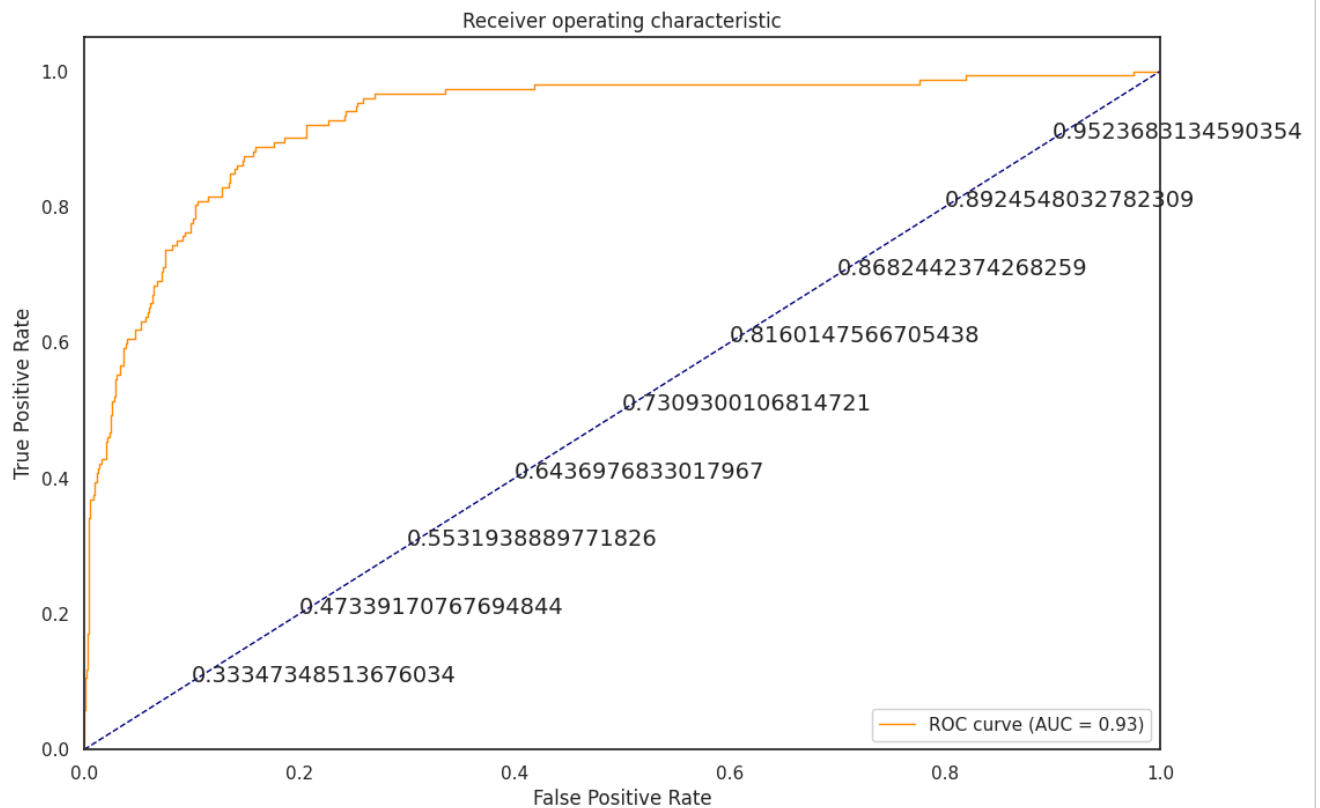
from sklearn.metrics import roc_curve, auc
fpr, tpr, thresholds = roc_curve(y_val, rfc.predict_proba(X_val)[:,-1])
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=1, label='ROC curve (AUC = %0.2f)' %
roc_auc)

for label in range(1,10,1):
    plt.text((10-label)/10,(10-label)/10,thresholds[label*15],fontdict={'size':
14})

```

```
plt.plot([0, 1], [0, 1], color='navy', lw=1, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```



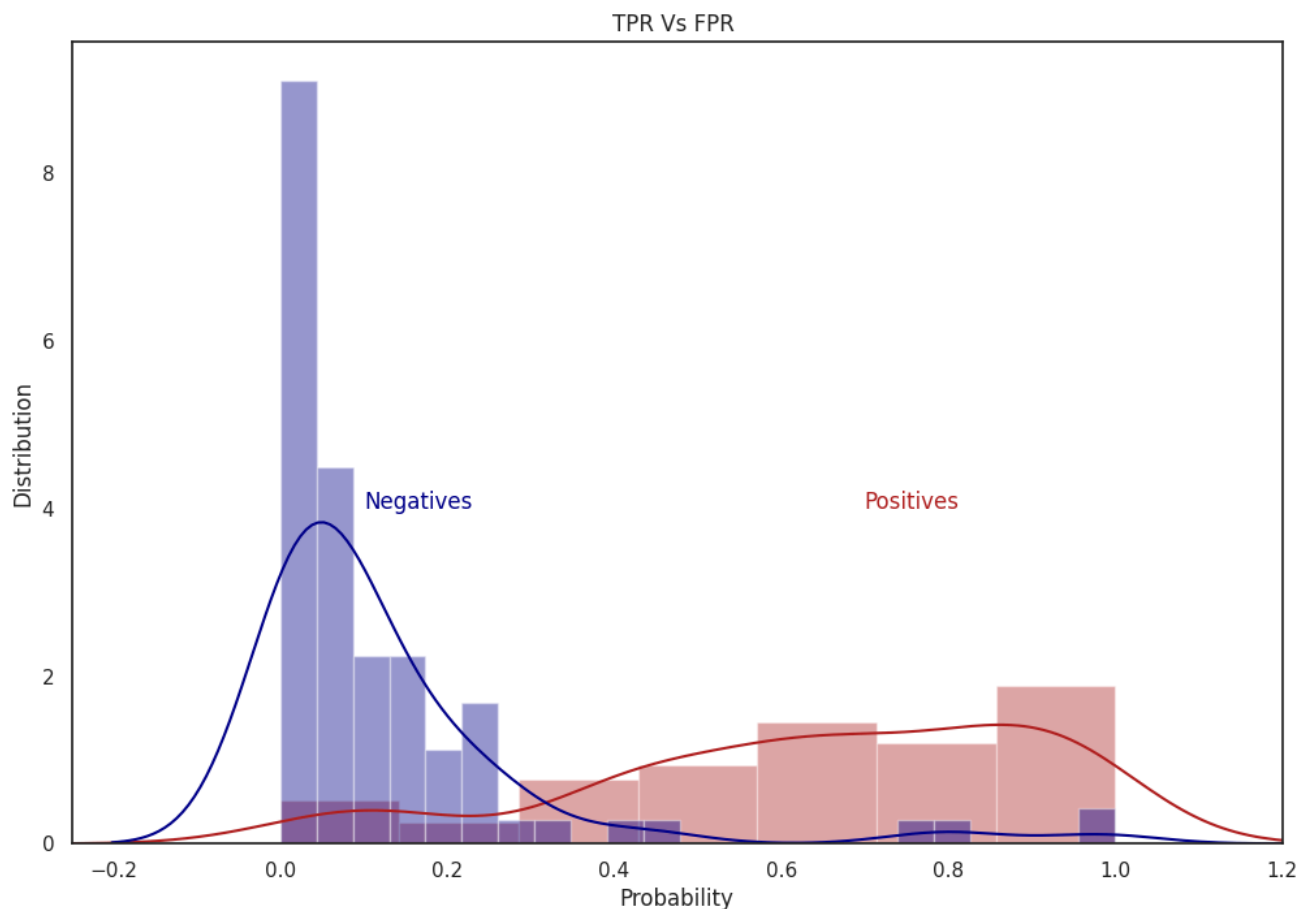
**** Random Forest : TPR Vs FPR****

```
fig = plt.figure(figsize=(12,8))

sns.distplot(tpr,color='firebrick')

sns.distplot(fpr,color='darkblue')
plt.title('TPR Vs FPR ')
plt.xlim([-0.25, 1.2])

plt.text(0.1,4,'Negatives',color='darkblue')
plt.text(0.7,4,'Positives',color='firebrick')
plt.xlabel('Probability')
plt.ylabel('Distribution')
plt.show()
```



```
rfc_train_pred = (rfc.predict_proba(X_train)[:,:1]>0.5).astype(bool) # Set threshold to 0.5
rfc_val_pred = (rfc.predict_proba(X_val)[:,:1]>0.5).astype(bool) # Set threshold to 0.5
```

Random Forest : Model Evaluation

```
#Confusion matrix, Accuracy, sensitivity and specificity
from sklearn.metrics import confusion_matrix, accuracy_score, cohen_kappa_score,
roc_auc_score, f1_score, roc_curve

cm0 = confusion_matrix(y_train, rfc_train_pred, labels=[1,0])
print('Confusion Matrix Train : \n', cm0)

cm1 = confusion_matrix(y_val, rfc_val_pred, labels=[1,0])
print('Confusion Matrix Test: \n', cm1)

total0=sum(sum(cm0))
total1=sum(sum(cm1))
#####from confusion matrix calculate accuracy
accuracy0=(cm0[0,0]+cm0[1,1])/total0
print ('Accuracy Train : ', accuracy0)

accuracy1=(cm1[0,0]+cm1[1,1])/total1
print ('Accuracy Test : ', accuracy1)

sensitivity1 = cm1[0,0]/(cm1[0,0]+cm1[0,1])
print('Sensitivity : ', sensitivity1 )

specificity1 = cm1[1,1]/(cm1[1,0]+cm1[1,1])
print('Specificity : ', specificity1)

KappaValue=cohen_kappa_score(y_val, rfc_val_pred)
print("Kappa Value :",KappaValue)
AUC=roc_auc_score(y_val, rfc_val_pred)
```



```
print("AUC : ",auc)

print("F1-Score Train",f1_score(y_train,rfc_train_pred))
print("F1-Score Validation : ",f1_score(y_val, rfc_val_pred))
```

```
Confusion Matrix Train :
[[ 319   35]
 [ 395 3038]]
Confusion Matrix Test:
[[ 124   28]
 [ 188 1283]]
Accuracy Train : 0.8864536572484817
Accuracy Test : 0.866913123844732
Sensitivity : 0.8157894736842105
Specificity : 0.8721957851801495
Kappa Value : 0.4674031940494422
AUC : 0.8439926294321801
F1-Score Train 0.5973782771535581
F1-Score Validation : 0.5344827586206896
```

Confusion Matrix Train : [[275 79] [239 3194]] Confusion Matrix Test: [[107 45] [105 1366]] Accuracy Train : 0.916028518616319 Accuracy Test : 0.9075785582255084 Sensitivity : 0.7039473684210527 Specificity : 0.9286199864038069 Kappa Value : 0.5374522157454344 AUC : 0.8162836774124298 F1-Score Train 0.6336405529953918 F1-Score Validation : 0.587912087912088

Random Forest : Feature Importance

```
feature_list = list(Test_category_removed_groupedbyProv_PF.columns)
# Get numerical feature importances
importances = list(rfc.feature_importances_)
# List of tuples with variable and importance
feature_importances = [(feature, round(importance, 2)) for feature, importance in zip(feature_list[1:], importances)]
# Sort the feature importances by most important first
feature_importances = sorted(feature_importances, key = lambda x: x[1], reverse = True)
# Print out the feature and importances
print('Top -20 features impacting Random forest model and their importance score :- \n',)
[print('Variable: {:20} Importance: {}'.format(*pair)) for pair in feature_importances[:15]];
```

Top -20 features impacting Random forest model and their importance score :-

```
Variable: InscClaimAmtReimbursed Importance: 0.08
Variable: PerProviderAvg_InscClaimAmtReimbursed Importance: 0.08
Variable: PerAttendingPhysicianAvg_InscClaimAmtReimbursed Importance: 0.07
Variable: PerOperatingPhysicianAvg_InscClaimAmtReimbursed Importance: 0.05
Variable: PerClnAdmitDiagnosisCodeAvg_DeductibleAmtPaid Importance: 0.05
Variable: PerClnAdmitDiagnosisCodeAvg_InscClaimAmtReimbursed Importance: 0.04
Variable: ClnCount_Provider_ClnDiagnosisCode_8 Importance: 0.04
Variable: AdmitForDays Importance: 0.03
Variable: PerOperatingPhysicianAvg_IPAnnualReimbursementAmt Importance: 0.03
Variable: PerClnDiagnosisCode_1Avg_DeductibleAmtPaid Importance: 0.03
Variable: PerClnDiagnosisCode_4Avg_DeductibleAmtPaid Importance: 0.03
Variable: ClnCount_Provider_ClnDiagnosisCode_7 Importance: 0.03
Variable: ClnCount_Provider_ClnDiagnosisCode_9 Importance: 0.03
Variable: DeductibleAmtPaid Importance: 0.02
Variable: PerOperatingPhysicianAvg_AdmitForDays Importance: 0.02
```

Random Forest : Prediction on Unseen Data

```
## Lets predict on Test data

rfc_test_pred = rfc.predict(X_teststd)
```

```
rfc_test_pred=pd.DataFrame(rfc_test_pred)
rfc_test_pred.head(2)
```

```
   0  
0  0
1  0
```

Next steps: [Generate code with rfc_test_pred](#) [New interactive sheet](#)

```
## Lets Replace 1 with Yes and 0 With No

Replacement = {1:'Yes',0:'No'}

Labels=rfc_test_pred[0].apply(lambda x : Replacement[x])
```

```
Labels.value_counts()
```

```

      count
0
No    1094
Yes    259

dtype: int64
```

```
submission_rfc=pd.DataFrame({"Provider":Test_category_removed_groupedbyProv_PF.Provider})
submission_rfc['PotentialFraud']=Labels
submission_rfc.shape
```

```
(1353, 2)
```

```
print('Shape Of Submission',submission_rfc.shape)
```

```
Shape Of Submission (1353, 2)
```

```
#write Submission file

submission_rfc.to_csv("Submission_Random_Forest_Estimator_F1_60_estimators_3000.csv",index=False)
```

Other Models and their Performance

```
## Lets import packages
from collections import Counter
from sklearn.naive_bayes import MultinomialNB

from sklearn.svm import SVC
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
import pprint
```

```
clfs = {
    #'mnb': MultinomialNB(),
    'svm1': SVC(C=0.01,kernel='linear',probability=True),
    'svm2': SVC(C=0.01,kernel='rbf',probability=True),
    'svm3': SVC(C=.01,kernel='poly',degree=2,probability=True),
    'ada': AdaBoostClassifier(),
    'dtc': DecisionTreeClassifier(class_weight='balanced'),
    'gbc': GradientBoostingClassifier(),
    'lr': LogisticRegression(class_weight='balanced'),
    'xgb': XGBClassifier(booster='gbtree')
}
```

```
## Lets Fit These models and check their performance
```

```
f1_scores = dict()
for clf_name in clfs:
    print(clf_name)
```

```
clf = clfs[clf_name]
clf.fit(X_train, y_train)
y_pred = ((clf.predict_proba(X_val)[:,1]>0.5).astype(bool))
f1_scores[clf_name] = f1_score(y_pred, y_val)
```

```
svm1
svm2
svm3
ada
dtc
gbc
lr
xgb
```

```
pprint.pprint(f1_scores) #Printing F1 Scores
```

```
{'ada': 0.6047904191616766,
'dtc': 0.4377104377104377,
'gbc': 0.5587044534412956,
'lr': 0.6091370558375635,
'svm1': 0.5067873303167421,
'svm2': 0.5619834710743802,
'svm3': 0.43349753694581283,
'xgb': 0.5381526104417671}
```

Above figures shows that Logistic regression model is performing better compared to other models.

Principal Component Analysis

```
Train_category_removed_groupedbyProv_PF.head(2)
Test_category_removed_groupedbyProv_PF.head(2)
```

	Provider	InscClaimAmtReimbursed	DeductibleAmtPaid	AdmitForDays	NoOfMonths_PartACov	NoOfMonths_PartBCov	ChronicCond_AIzI
0	PRV51002	53790	380.0	0.0	2424	2460	
1	PRV51006	30720	0.0	0.0	1224	1224	

2 rows × 153 columns

```
# Apply Standard Scaler to scale data
sc_pca=StandardScaler()
sc_pca.fit(Train_category_removed_groupedbyProv_PF.iloc[:,2:])
Train_category_removed_groupedbyProv_PF_scaled=sc_pca.transform(Train_category_removed_groupedbyProv_PF.iloc[:,2:])
Train_category_removed_groupedbyProv_PF_scaled=pd.DataFrame(Train_category_removed_groupedbyProv_PF_scaled)

Test_category_removed_groupedbyProv_PF_scaled=sc_pca.transform(Test_category_removed_groupedbyProv_PF.iloc[:,1:])
Test_category_removed_groupedbyProv_PF_scaled=pd.DataFrame(Test_category_removed_groupedbyProv_PF_scaled)
```

```
Train_category_removed_groupedbyProv_PF_scaled.shape
```

```
(5410, 152)
```

```
Test_category_removed_groupedbyProv_PF_scaled.shape
```

```
(1353, 152)
```

PCA : Use maximum Variance

```
from sklearn.decomposition import PCA #import PCA
```

```
# Lets Take 29 PCA components
pca = PCA(n_components=29)
```

```
pca.fit(Train_category_removed_groupedbyProv_PF_scaled.iloc[:,0:])
```

PCA (i ?)

PCA(n_components=29)

```
print('PCA Explained Variance :\n',np.round(pca.explained_variance_ratio_,3))
```

```
PCA Explained Variance :
[0.689 0.22  0.023 0.015 0.01  0.007 0.005 0.004 0.004 0.003 0.002 0.002
 0.002 0.002 0.001 0.001 0.001 0.001 0.001 0.001 0.001 0.001 0.    0.
 0.    0.    0.    0.    0.    ]
```

```
## Transform test and train based on train components
train_pca=pca.transform(Train_category_removed_groupedbyProv_PF_scaled.iloc[:,0:])

test_pca=pca.transform(Test_category_removed_groupedbyProv_PF_scaled)
```

```
print('Train Shape:',train_pca.shape)
print('Test Shape:',test_pca.shape)
```

```
Train Shape: (5410, 29)
Test Shape: (1353, 29)
```

```
## Convert to Pandas dataframe
```

```
train_pca=pd.DataFrame((train_pca))
test_pca=pd.DataFrame(test_pca)
```

```
print('Train Shape:',train_pca.shape)
print('Test Shape:',test_pca.shape)
```

```
Train Shape: (5410, 29)
Test Shape: (1353, 29)
```

```
#Add target to train Data
train_pca['PotentialFraud']=Train_category_removed_groupedbyProv_PF.PotentialFraud
```

```
train_pca.isnull().sum().sum()
print('Train PCA',train_pca.shape)
#train_pca.head()
print('Test PCA',test_pca.shape)
```

```
Train PCA (5410, 30)
Test PCA (1353, 29)
```

```
train_pca.head(2)
```

	0	1	2	3	4	5	6	7	8	9	...	20	21
0	-2.351058	0.352104	0.155031	-0.587395	0.195851	0.098480	-0.206957	-0.291128	-0.054530	-0.233879	...	0.107890	-0.155001
1	9.208414	9.685606	-0.715757	-4.481318	-4.210365	0.032422	0.405085	-0.276264	0.346817	0.136236	...	0.083556	-0.460963

2 rows × 30 columns

Auto - Encoder

****An Autoencoder is a type of artificial neural network used to learn efficient data codings in an unsupervised manner. The aim of an autoencoder is to learn a representation (encoding) for a set of data, typically for dimensionality reduction. Along with the reduction side, a reconstructing side is learnt, where the autoencoder tries to generate from the reduced encoding a representation as close as possible to its original input, hence its name. Recently, the autoencoder concept has become more widely used for learning generative models of data.**

**** We will use the same technique to learn patterns in non fraud data and we will train the model accordingly. We will use reconstruction error threshold ,to predict the Target class of data.****

```
from sklearn.model_selection import train_test_split
#Converting data to array
```

```
train_pca = np.array(train_pca)
```

```
#Splitting the data into train and test and observing their dimensions
X_train_pca, X_test_pca = train_test_split(train_pca, test_size=0.2, random_state=123)

print(X_train_pca.shape)
print(X_test_pca.shape)
print(test_pca.shape)
```

```
(4328, 30)
(1082, 30)
(1353, 29)
```

```
#Obtaining the fraud and non-fraud records in train
print(np.unique(X_train_pca[:,29],return_counts=True))
print(np.unique(X_test_pca[:,29],return_counts=True))
```

```
(array([0., 1.]), array([3916, 412]))
(array([0., 1.]), array([988, 94]))
```

We will consider Non fraud data to train the autoencoder and check the threshold for reconstruction error on fraud data.

```
#Now consider only the non-fraud records for training
X_train_NF = X_train_pca[X_train_pca[:, -1] == 0]
X_train_NF = X_train_NF[:, :-1]
print(X_train_NF.shape)
```

```
(3916, 29)
```

```
#Separating out the fraud records from the train
X_train_F = X_train_pca[X_train_pca[:, -1] == 1]
print(X_train_F.shape)
```

```
(412, 30)
```

```
#Adding/concatenating the fraud records from train data to the test
X_test_pca=np.concatenate((X_test_pca,X_train_F),axis=0)
print(X_test_pca.shape)
```

```
(1494, 30)
```

```
X_test,X_eval = train_test_split(X_test_pca, test_size=0.2, random_state=123)
```

```
print(X_test.shape)
print(X_eval.shape)
```

```
(1195, 30)
(299, 30)
```

```
#Separating the independent and the class variable
y_test = X_test[:, -1]
X_test = X_test[:, :-1]
```

```
#Expanding the dimensions of y for later concatenation
y_test = np.expand_dims(y_test, axis=1)
y_test.shape
```

```
(1195, 1)
```

Autoencoder Network Building

```
input_dim = X_train_NF.shape[1] # Set input dimension to number of inputs
encoding_dim = 15 # set encoding dimension to size 15
```

```
from keras.models import Sequential
from keras.layers import Dense, Dropout

autoencoder = Sequential() # Add sequential network
```

```

autoencoder.add(Dropout(0.2, input_shape=(input_dim,))) # Add dropout to add noise to data.
autoencoder.add(Dense(encoding_dim, activation='relu')) #Add Dense layer to encode patterns.
autoencoder.add(Dense(input_dim, activation='linear')) #Add Dense layer of size of input shape

```

```

nb_epoch = 100    #number of epochs
batch_size = 32   #batchsize or sample size

autoencoder.compile(optimizer='adam',
                    loss='mean_squared_error',
                    metrics=['mse']) ## We will use mse as a metric as each neuron can be a linear regress

```

```

## Fit the model and save history
hist = []
for _ in range(100):
    hist.append(autoencoder.fit(X_train_NF, X_train_NF,
                              epochs=1,
                              batch_size=batch_size,
                              shuffle=True,
                              validation_split=0.3,
                              verbose=0).history)

```

```
hist[0:5]    #First 5 records of history
```

```

[{'loss': [1.2300400733947754],
  'mse': [1.2300400733947754],
  'val_loss': [0.7353728413581848],
  'val_mse': [0.7353728413581848]},
 {'loss': [0.8175879120826721],
  'mse': [0.8175879120826721],
  'val_loss': [0.5105670094490051],
  'val_mse': [0.5105670094490051]},
 {'loss': [0.5962545275688171],
  'mse': [0.5962545275688171],
  'val_loss': [0.37671032547950745],
  'val_mse': [0.37671032547950745]},
 {'loss': [0.4503718614578247],
  'mse': [0.4503718614578247],
  'val_loss': [0.3015933632850647],
  'val_mse': [0.3015933632850647]},
 {'loss': [0.4021255671977997],
  'mse': [0.4021255671977997],
  'val_loss': [0.258475661277771],
  'val_mse': [0.258475661277771]}]

```

```

## Making predictions on the train data
predictions=autoencoder.predict(X_train_NF)

```

```
123/123 ————— 0s 2ms/step
```

```
predictions[0:2]    #See the predictions
```

```

array([[ -3.89071894e+00,  -4.00201529e-01,   4.78269398e-01,
        -7.25272298e-02,   8.63049850e-02,   2.64197960e-02,
         1.90366864e-01,   1.21329665e-01,   4.83936667e-02,
        -2.48091221e-02,   3.59874964e-03,  -2.27588415e-03,
        -9.80859995e-03,  -4.23163474e-02,  -5.02832383e-02,
         1.09284259e-02,  -3.94444168e-03,  -2.62677968e-02,
        -1.63933672e-02,   8.12660158e-03,  -1.32806599e-03,
        -1.93148851e-04,   8.26001167e-03,   2.28055716e-02,
        -1.19982660e-03,  -3.19923460e-03,   1.54453684e-02,
         8.23158398e-03,  -7.49748200e-03],
       [ -2.66281962e+00,   1.56138372e+00,   9.35705125e-01,
         1.43956447e+00,   4.41808105e-01,  -5.20610094e-01,
        -1.02091581e-01,  -4.11389768e-02,  -7.21335948e-01,
         9.95451510e-02,  -9.47363973e-02,   1.01616383e-02,
        -2.70619839e-02,  -5.62237501e-02,   3.42213064e-02,
        -8.48777518e-02,   6.92538098e-02,  -1.19834848e-01,
         2.24691518e-02,  -2.54905708e-02,   2.06036568e-02,
         2.33750045e-02,  -3.02634463e-02,  -1.10096931e-02,
        -4.35708463e-03,   2.36940533e-02,  -8.55716038e-03,
        -4.40118685e-02,  -5.14243543e-02]], dtype=float32)

```

```
##We want to separate out fraud records and non-fraud records for later use
f = np.hstack((X_test,y_test))
print(f.shape)

test_nf=f[f[:,29]==0]
print('Test_nf',test_nf.shape)

test_f=f[f[:,29]==1]
print('Test_f',test_f.shape)
```

```
(1195, 30)
Test_nf (788, 30)
Test_f (407, 30)
```

```
#Getting the errors from the non fraud data separately
autoencoder.evaluate(test_nf[:, :29], test_nf[:, :29])
```

```
25/25 ————— 0s 3ms/step - loss: 0.0760 - mse: 0.0760
[0.07598449289798737, 0.07598449289798737]
```

```
#Getting the errors from the fraud data separately
autoencoder.evaluate(test_f[:, :29], test_f[:, :29])
```

```
13/13 ————— 0s 3ms/step - loss: 7.4104 - mse: 7.4104
[7.410360336303711, 7.410360336303711]
```

```
#Obtaining predictions for non fraud records
predictions_nf=autoencoder.predict(test_nf[:, :29])
```

```
25/25 ————— 0s 2ms/step
```

```
#Obtaining predictions for fraud records
predictions_f=autoencoder.predict(test_f[:, :29])
```

```
13/13 ————— 0s 4ms/step
```

```
#Identifying the error computation method by autoencoder(Mean Squared Error). The computation is as follows
np.mean(np.square(np.abs(test_nf[:, :29]-predictions_nf)))
```

```
np.float64(0.07598448559612152)
```

```
#Computing errors on the non-fraud data
errors_nf = np.mean(np.square(np.abs(test_nf[:, :29]-predictions_nf)), axis=1)
```

```
errors_nf[0:5]
```

```
array([0.00135887, 0.00058759, 0.00805936, 0.00114292, 0.00200026])
```

```
#Computing errors on the fraud data
errors_f = np.mean(np.square(np.abs(test_f[:, :29]-predictions_f)), axis=1)
```

```
errors_f[0:5]
```

```
array([ 0.08548679,  0.17379947,  0.42374034,  2.95311778, 11.50453478])
```

```
#Computing the distribution of errors in both non-fraud and fraud data
print(np.min(errors_nf))
print(np.max(errors_nf))
print(np.median(errors_nf))

print(np.min(errors_f))
print(np.max(errors_f))
print(np.median(errors_f))
```

```
0.00041007970112740677
25.690032122284304
0.005131030693627409
0.0006650530817670315
547.1841254076379
0.3272735018218205
```

```
0.24613738200955018
```

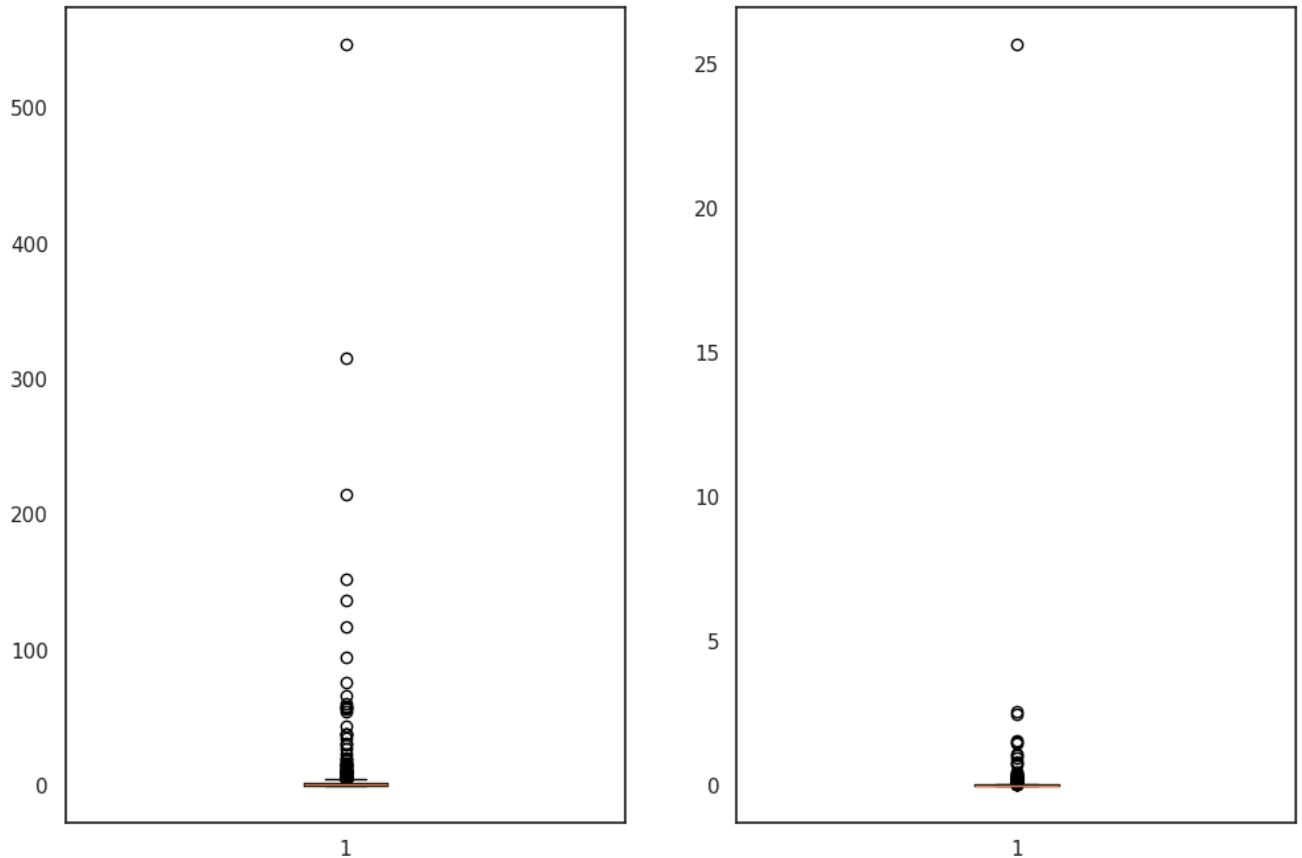
```
#Plotting the error box plots
```

```
plt.subplot(1, 2,1)
plt.boxplot(errors_f)
```

```
plt.subplot(1, 2, 2)
plt.boxplot(errors_nf)
```

```
fig.suptitle('Error boxplot for Fraud and nonfraud')
```

```
Text(0.5, 0.98, 'Error boxplot for Fraud and nonfraud')
```



```
#Experimentation to fix a threshold for classification of a transaction into fraud or non-fraud
```

```
print(sum(errors_nf>np.median(errors_f)))
print(sum(errors_f<np.median(errors_f)))
print(sum(errors_f<np.median(errors_nf)))
print(sum(errors_nf>np.median(errors_nf)))
```

```
18
203
5
394
```

```
print(errors_nf.shape)
print(errors_f.shape)
```

```
(788,)
(407,)
```

```
print(predictions_nf.shape)
print(predictions_f.shape)
```

```
(788, 29)
(407, 29)
```

```
test_pred = autoencoder.predict(X_test[:, :29])
test_recon = (((test_pred-X_test)**2).mean(-1))
```

```
train_pred = autoencoder.predict(X_train_NF[:, :29])
mean_recon = (((train_pred - X_train_NF)**2).mean(-1).mean())
```



```

from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix

scores_f1 = []
thres = []

th = 0
for i in range(100):
    th+=0.1
    fraud = (test_recon>mean_recon+th)
    scores_f1.append(f1_score(y_test,fraud))
    thres.append(th+mean_recon)

```

38/38 ————— 0s 2ms/step
 123/123 ————— 0s 1ms/step

X_test.shape

(1195, 29)

```

plt.plot(thres, scores_f1)

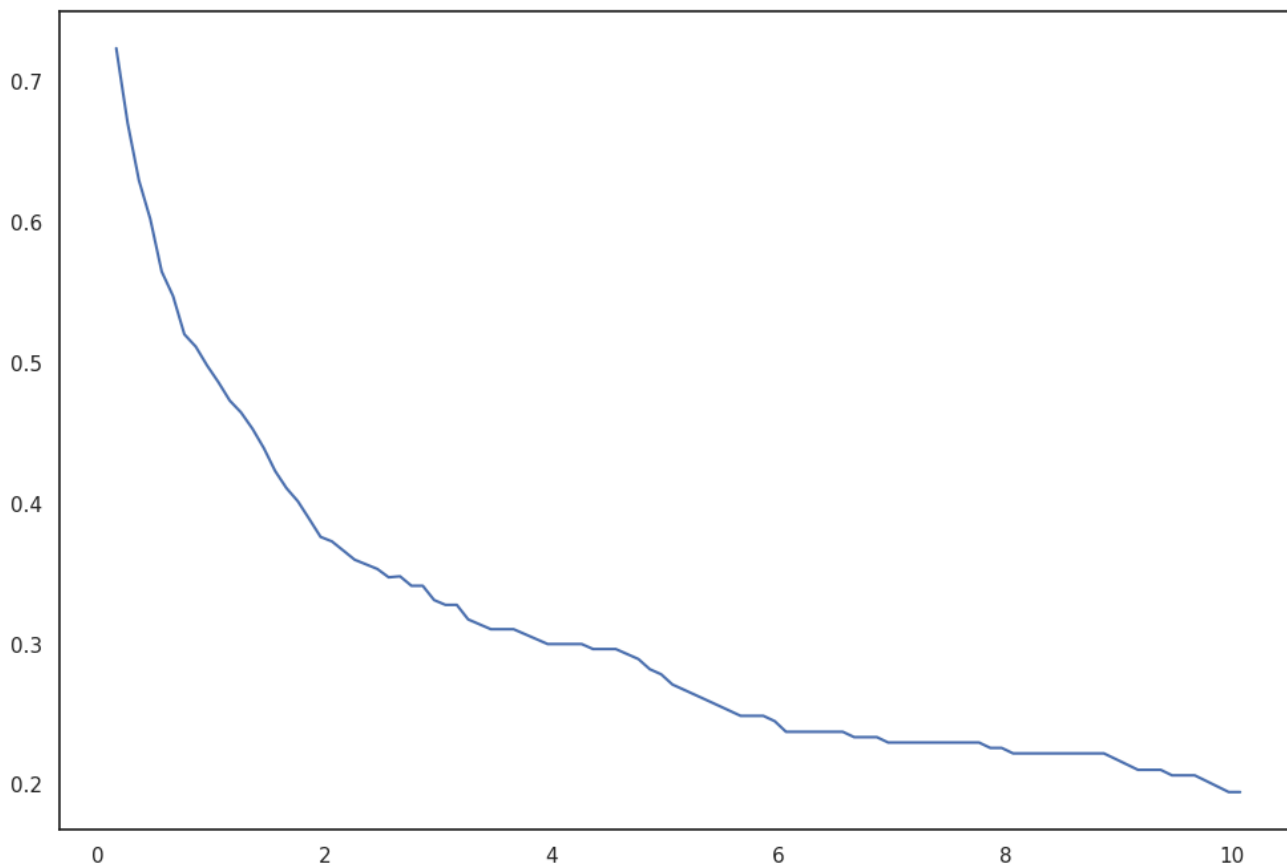
print(thres[np.array(scores_f1).argmax()])

fraud = (test_recon>thres[np.array(scores_f1).argmax()])

confusion_matrix(y_test, fraud)

```

0.16773590663245525
 array([[749, 39],
 [154, 253]])



##Predicting on Validation

```
predictions_eval=autoencoder.predict(X_eval[:, :29])
```

10/10 ————— 0s 3ms/step

```
errors_eval=np.square(np.subtract(predictions_eval,X_eval[:, :29]))
```

```
threshold=0.1718219230420997
```

```
fraud_eval=((errors_eval-X_eval[:, :29])**2).mean(-1)>threshold
```

```
print(confusion_matrix(y_true=X_eval[:, 29],y_pred=fraud_eval,labels=[1,0]))
```

```
[[ 99   0]
 [190  10]]
```

```
#Obtaining errors on individual attributes on non-fraud data
ind_errors_nf=np.abs(np.subtract(predictions_nf,test_nf[:, :29]))
print(ind_errors_nf.shape)
```

```
(788, 29)
```

```
#Obtaining errors on individual attributes on fraud data
ind_errors_f=np.abs(np.subtract(predictions_f,test_f[:, :29]))
print(ind_errors_f.shape)
```

```
(407, 29)
```

```
#Combining all fraud and non-fraud data errors
X=np.concatenate((ind_errors_f,ind_errors_nf),axis=0)
print(X.shape)
```

```
(1195, 29)
```

```
#Creating a target column
x=np.array([1,0])
Y=np.repeat(x,[788,407])
```

```
#Expanding the shape of the target for merging with all other attributes
print(Y.shape)
Y = np.expand_dims(Y, axis=1)
print(Y.shape)
```

```
(1195,)
(1195, 1)
```

```
#Concatenating all the attribute errors with target class
X=np.concatenate((X,Y),axis=1)
print(X.shape)
```

```
(1195, 30)
```

```
map(str,range(X.shape[1]))
```

```
<map at 0x7f56fcc74430>
```

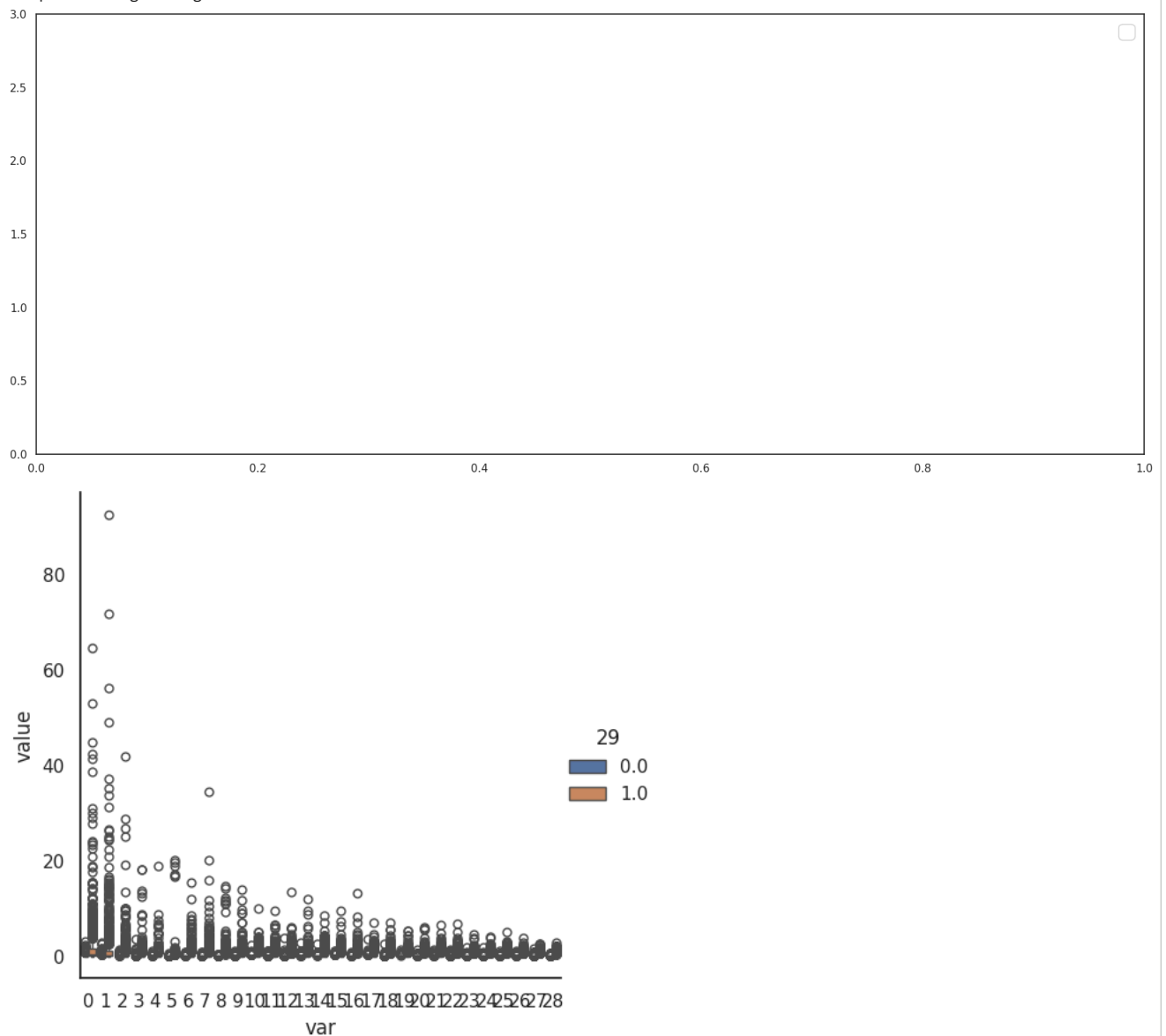
```
#Creating a data frame and naming the columns
X=pd.DataFrame(X)
X.columns = map(str, range(X.shape[1]))
```

```
#Melting the data for box plot visualization
M = pd.melt(X, "29", var_name="var", value_name="value")
M['29'].astype('category')
M.tail()
#M.head()
```

	29	var	value
34650	0.0	28	0.146504
34651	0.0	28	0.003933
34652	0.0	28	0.015957
34653	0.0	28	0.239011
34654	0.0	28	0.149755

```
import seaborn as sns
size=(20,8)
fig,ax = plt.subplots(figsize=size)
# the size of A4 paper
g=sns.catplot(x="var", y='value', hue="29", data=M, kind='box',ax=ax)
ax.set_yscale(value="linear")
ax.set_ybound(0,3)
ax.legend(fontsize=20)
```

<matplotlib.legend.Legend at 0x7f56f407c9b0>



```
pred_nf=ind_errors_nf[:,[0,1,11,17,18,4,8,14]]
np.mean(pred_nf,axis=0)
```

```
array([0.32296333, 0.16581096, 0.07949396, 0.12420773, 0.06709434,  
       0.09649062, 0.06959877, 0.09351973])
```

```
pred_f=ind_errors_f[:,[0,1,11,17,18,4,8,14]]  
np.mean(pred_f,axis=0)
```

```
array([3.92222301, 3.68111446, 0.6261393 , 0.6405904 , 0.52088535,  
       0.63132024, 0.65166224, 0.63391439])
```

```
#Percentile distribution of absolute errors  
percentile_nf=np.percentile(ind_errors_nf[:,[0,1,11,17,18,4,8,14]], [10,20,30,40,50,60,70,80,90,100],axis=0)
```

```
percentile_nf
```

```
array([[3.39341135e-02, 1.84081423e-02, 8.53782750e-03, 6.16745666e-03,  
       4.16251190e-03, 1.14132357e-02, 9.48526317e-03, 3.79125874e-03],  
       [7.40509976e-02, 3.63589167e-02, 1.60226771e-02, 1.24244075e-02,  
       7.43407164e-03, 2.00276699e-02, 1.86066580e-02, 8.44128150e-03],  
       [1.10264881e-01, 4.88227373e-02, 2.24822385e-02, 2.13964599e-02,  
       1.05551978e-02, 2.46318336e-02, 2.66087916e-02, 1.62689646e-02],  
       [1.36964826e-01, 6.38543160e-02, 2.85400112e-02, 3.28732171e-02,  
       1.52989591e-02, 2.95542525e-02, 3.30587106e-02, 2.37593241e-02],  
       [1.56791452e-01, 7.77105435e-02, 3.34508843e-02, 5.04571894e-02,  
       2.22185797e-02, 3.77191480e-02, 3.76548369e-02, 3.85751594e-02],  
       [2.08118325e-01, 9.97642316e-02, 4.95316185e-02, 8.04992589e-02,  
       3.37082419e-02, 5.44641883e-02, 4.15972374e-02, 5.32431776e-02],  
       [2.87862157e-01, 1.29303441e-01, 7.24059727e-02, 1.16470272e-01,  
       5.06458628e-02, 8.41450355e-02, 4.58405715e-02, 7.57739088e-02],  
       [4.33353685e-01, 1.68879531e-01, 1.05092563e-01, 1.95872885e-01,  
       8.15997400e-02, 1.31641536e-01, 6.14168970e-02, 1.17221273e-01],  
       [7.50115216e-01, 2.73959125e-01, 1.80692505e-01, 2.98721907e-01,  
       1.49211084e-01, 2.32322853e-01, 1.06886743e-01, 1.83569870e-01],  
       [5.90302922e+00, 5.48664219e+00, 1.16025531e+00, 3.61155846e+00,  
       2.61710355e+00, 6.75640214e+00, 1.10983301e+01, 4.02083845e+00]])
```

```
percentile_nf[:,7]
```

```
array([3.79125874e-03, 8.44128150e-03, 1.62689646e-02, 2.37593241e-02,  
       3.85751594e-02, 5.32431776e-02, 7.57739088e-02, 1.17221273e-01,  
       1.83569870e-01, 4.02083845e+00])
```

```
percentile_f=np.percentile(ind_errors_f[:,[0,1,11,17,18,4,8,14]], [10,20,30,],  
axis=0)  
np.percentile(ind_errors_f[:,[0,1,11,17,18,4,8,14]], [10,20,30,40,50,60,70,80,90,  
100],axis=0)
```

```
array([[1.61931785e-01, 8.27705449e-02, 3.73803165e-02, 5.82634456e-02,  
       3.07981980e-02, 4.60533857e-02, 2.93235096e-02, 3.41837300e-02],  
       [3.73547012e-01, 1.76159885e-01, 7.99844142e-02, 1.16413786e-01,  
       5.77280602e-02, 1.01447647e-01, 5.56730817e-02, 9.29561387e-02],  
       [5.32173375e-01, 2.61695041e-01, 1.26261704e-01, 1.98406137e-01,  
       9.38757337e-02, 1.46191293e-01, 9.08995515e-02, 1.57904626e-01],  
       [8.27418468e-01, 5.02684388e-01, 2.01693427e-01, 2.79665660e-01,  
       1.46806106e-01, 2.27691728e-01, 1.32357769e-01, 2.27701938e-01],  
       [1.18504982e+00, 8.28422743e-01, 2.96349314e-01, 4.13632474e-01,  
       2.19759695e-01, 3.31436570e-01, 1.75089312e-01, 3.08783886e-01],  
       [2.06420587e+00, 1.33662563e+00, 3.91369942e-01, 5.08348381e-01,  
       3.20148324e-01, 4.38995562e-01, 2.49331996e-01, 4.32206247e-01],  
       [3.37052226e+00, 2.33671843e+00, 6.00597264e-01, 7.58109644e-01,  
       4.48475569e-01, 5.75608503e-01, 3.89825814e-01, 6.13790381e-01],  
       [5.46699389e+00, 4.24044284e+00, 9.38399712e-01, 9.96741844e-01,  
       7.80022861e-01, 7.59249903e-01, 6.46029629e-01, 8.87030935e-01],  
       [9.45751029e+00, 9.59301514e+00, 1.49725339e+00, 1.44926259e+00,  
       1.23504814e+00, 1.21992377e+00, 1.35316898e+00, 1.52332291e+00],  
       [6.48512972e+01, 9.28070906e+01, 9.52983935e+00, 6.99883298e+00,  
       6.96376159e+00, 1.89656150e+01, 1.47116330e+01, 8.66179710e+00]])
```

```
percentile_f[:,7]
```

```
array([0.03418373, 0.09295614, 0.15790463])
```

Set Threshold

```
thr= 0.10003411    #0.50795386    #0.32079228
```

```
test_pred = autoencoder.predict(X_test)
```

38/38 ————— 0s 2ms/step
(1195, 29)

```
recon_error= np.abs(np.subtract(test_pred,X_test[:, :29]))
```

```
recon_error.shape
```

```
(1195, 29)
```

```
from sklearn.metrics import precision_score,recall_score,accuracy_score,confusion_matrix
fraud = (recon_error[:,1]>thr)
print("Recall=",recall_score(y_test,fraud))
print("Precision=",precision_score(y_test,fraud))
print("Accuracy=",accuracy_score(y_test,fraud))
print("F1-Score",f1_score(y_test,fraud))
```

```
Recall= 0.8796068796068796
Precision= 0.5327380952380952
Accuracy= 0.6962343096234309
F1-Score 0.6635773864689527
```

```
print(confusion_matrix(y_true=y_test,y_pred=fraud,labels=[1,0]))
```

```
[[358  49]
 [314 474]]
```

Evaluaton

```
predictions_eval=autoencoder.predict(X_eval[:, :29])
predictions_eval.shape
```

10/10 ————— 0s 5ms/step
(299, 29)

```
ind_errors_eval=np.abs(np.subtract(predictions_eval,X_eval[:, :29]))
ind_errors_eval.shape
```

```
(299, 29)
```

```
fraud_eval=ind_errors_eval[:,1]>thr
print("Recall",recall_score(X_eval[:, :29],fraud_eval))
print("Precision",precision_score(X_eval[:, :29],fraud_eval))
print("Accuracy",accuracy_score(X_eval[:, :29],fraud_eval))
print("F1-Score",f1_score(X_eval[:, :29],fraud_eval))
```

```
Recall 0.8686868686868687
Precision 0.5088757396449705
Accuracy 0.6789297658862876
F1-Score 0.6417910447761194
```

```
print(confusion_matrix(y_true=X_eval[:, :29],y_pred=fraud_eval,labels=[1,0]))
```

```
[[ 86  13]
 [ 83 117]]
```

```
fraud_eval[:6]
```

```
array([ True, False, False, False, False, False])
```

****Let's predict on unseen data ****

```
predictions_unseen=autoencoder.predict(test_pca[:, :29])
predictions_unseen.shape
```

43/43 ————— 0s 2ms/step
(1353, 29)

```
ind_errors_unseen=np.abs(np.subtract(predictions_unseen,test_pca[:, :29]))
ind_errors_unseen.shape
```

(1353, 29)

```
fraud_unseen=ind_errors_unseen[:,1]>thr
```

```
fraud_unseen.shape
```

(1353,)

```
fraud_unseen[:5]
```

```
array([ True,  True, False, False, False])
```

Evaluate results with the best performing model

```
print("Recall",recall_score((log.predict_proba(X_teststd)[: ,1]>0.60).astype(bool),fraud_unseen))
print("Precision",precision_score((log.predict_proba(X_teststd)[: ,1]>0.60).astype(bool),fraud_unseen))
print("Accuracy",accuracy_score((log.predict_proba(X_teststd)[: ,1]>0.60).astype(bool),fraud_unseen))
print("F1-Score",f1_score((log.predict_proba(X_teststd)[: ,1]>0.60).astype(bool),fraud_unseen))
```

Recall 0.9058823529411765
Precision 0.23839009287925697
Accuracy 0.6245380635624538
F1-Score 0.37745098039215685

```
fraud_unseen=pd.DataFrame(fraud_unseen)
```

```
fraud_unseen.shape
fraud_unseen.head(2)
```

	0	1
0	True	
1	True	

Next steps: [Generate code with fraud_unseen](#) [New interactive sheet](#)

```
Replacement = {1:'Yes',0:'No'}

AE_Labels=fraud_unseen[0].apply(lambda x : Replacement[x])
```

```
AE_Labels.value_counts()
```

	count
0	
No	707
Yes	646

dtype: int64

```
submission_AutoEncoder=pd.DataFrame
({"Provider":Test_category_removed_groupedbyProv_PF.Provider})
submission_AutoEncoder['PotentialFraud']=AE_Labels
submission_AutoEncoder.head(16)
```

	Provider	PotentialFraud
0	PRV51002	Yes
1	PRV51006	Yes
2	PRV51009	No
3	PRV51010	No
4	PRV51018	No
5	PRV51019	No
6	PRV51020	Yes
7	PRV51022	Yes
8	PRV51028	No
9	PRV51033	No
10	PRV51034	Yes
11	PRV51039	Yes
12	PRV51050	Yes
13	PRV51051	Yes
14	PRV51069	Yes
15	PRV51073	Yes

Next steps:

Generate code with submission_AutoEncoder

New interactive sheet

```
submission_AutoEncoder.shape
```

```
(1353, 2)
```

Autoencoder with 2 hidden layers

```
df = pd.DataFrame(train_pca) #Again use train
df.shape
```

```
(5410, 30)
```

```
df.head()
```

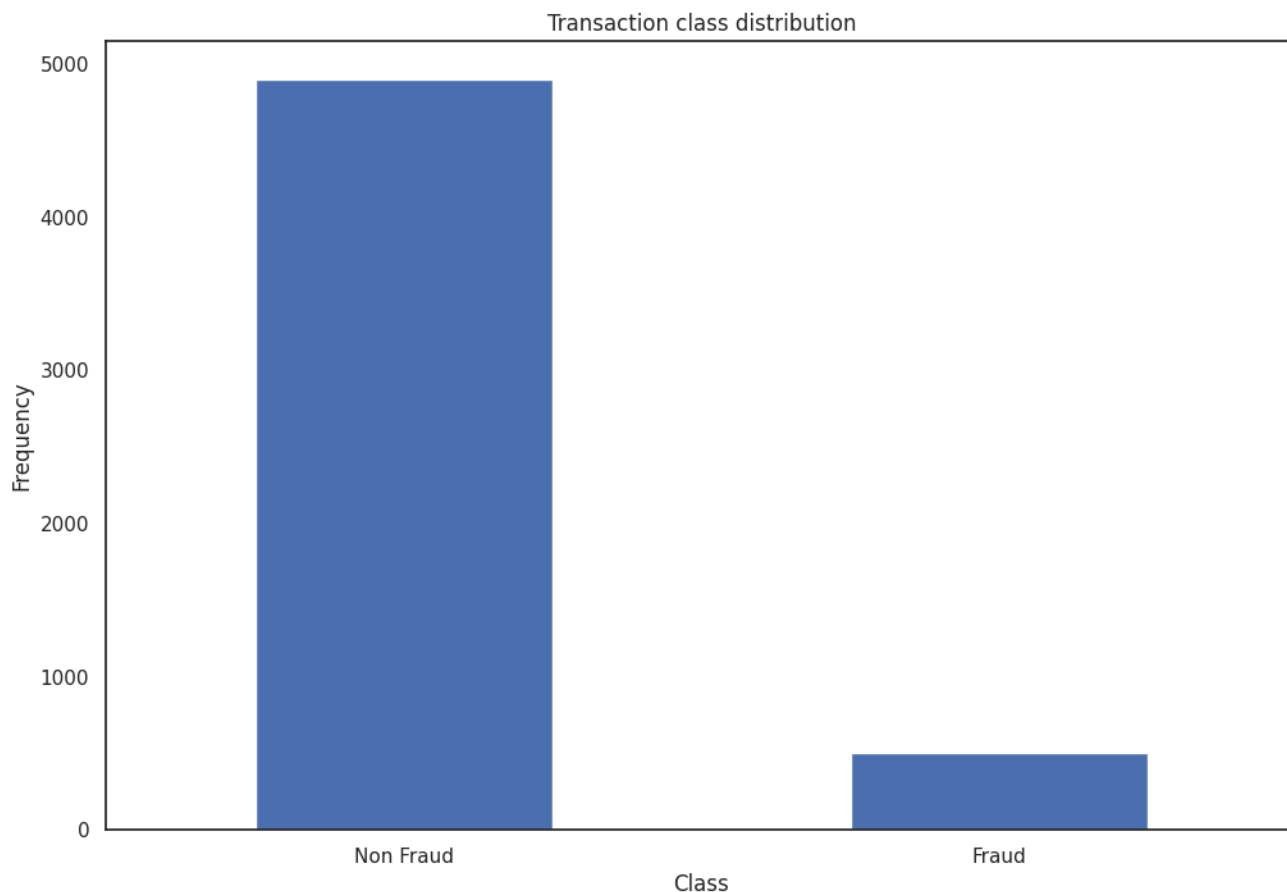
	0	1	2	3	4	5	6	7	8	9	...	20	21
0	-2.351058	0.352104	0.155031	-0.587395	0.195851	0.098480	-0.206957	-0.291128	-0.054530	-0.233879	...	0.107890	-0.155001
1	9.208414	9.685606	-0.715757	-4.481318	-4.210365	0.032422	0.405085	-0.276264	0.346817	0.136236	...	0.083556	-0.460963
2	-0.082284	-2.032673	-0.604175	0.148650	0.014296	-0.029213	-0.117141	0.224145	0.069811	0.018314	...	0.113068	-0.010830
3	26.698112	-15.557447	-4.516611	1.773201	-1.150199	-0.246206	-2.404637	0.897333	0.646572	0.139883	...	-0.314664	-0.352444
4	-2.134478	-0.758859	-0.001991	-0.103458	-0.107217	-0.020505	0.298905	0.227413	0.003481	0.063458	...	0.022999	0.075420

5 rows × 30 columns

```
df.isnull().values.any() #check null values
```

```
np.False_
```

```
# Lets check distruibution of classes
count_classes = pd.value_counts(df.iloc[:,29], sort = True)
count_classes.plot(kind = 'bar', rot=0)
plt.title("Transaction class distribution")
plt.xticks(range(2), LABELS)
plt.xlabel("Class")
plt.ylabel("Frequency");
```



```
frauds = df[df.iloc[:,29] == 1]
normal = df[df.iloc[:,29] == 0]
print('Fraud data shape:',frauds.shape)
```

```
Fraud data shape: (506, 30)
```

```
print('Normal data shape:',normal.shape)
```

```
Normal data shape: (4904, 30)
```

```
X_train_AE, X_test_AE = train_test_split(df, test_size=0.2, random_state=RANDOM_SEED)
X_train_AE = X_train_AE[X_train_AE.iloc[:,29] == 0]
X_train_AE = X_train_AE.drop([29], axis=1)
y_test_AE = X_test_AE[29]
X_test_AE = X_test_AE.drop([29], axis=1)
X_train_AE = X_train_AE.values
X_test_AE = X_test_AE.values
X_train_AE.shape
```

```
###df[ df.columns[[1,3]] ]
```

```
(3927, 29)
```

****Add 2 hidden layers to autoencoder ****

```
## AutoEncoder with 2 Hidden Layers
```

```
input_dim = X_train_AE.shape[1]
encoding_dim = 14
```

```
input_layer = Input(shape=(input_dim, ))
encoder = Dense(encoding_dim, activation="tanh",
                activity_regularizer=regularizers.l1(10e-5))(input_layer)
```



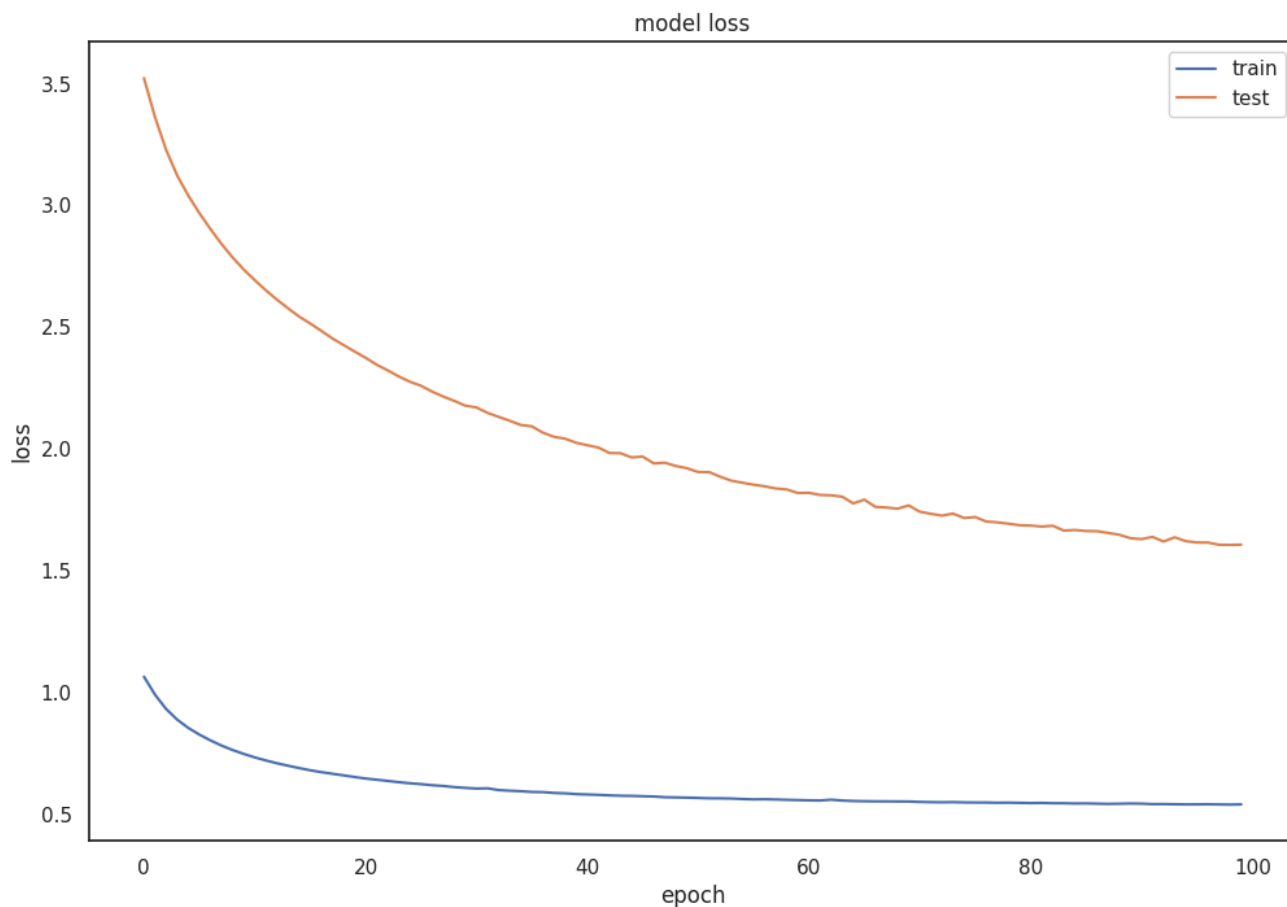
```
encoder = Dense(int(encoding_dim / 2), activation="relu")(encoder)
decoder = Dense(int(encoding_dim / 2), activation='tanh')(encoder)
decoder = Dense(input_dim, activation='relu')(decoder)
```

```
nb_epoch = 100
batch_size = 32
autoencoder.compile(optimizer='adam',
                    loss='mean_squared_error',
                    metrics=['accuracy'])
checkpointer = ModelCheckpoint(filepath="model.h5",
                              verbose=0,
                              save_best_only=True)
tensorboard = TensorBoard(log_dir='./logs',      #Tensorboard
                          histogram_freq=0,
                          write_graph=True,
                          write_images=True)
history = autoencoder.fit(X_train_AE, X_train_AE,
                        epochs=nb_epoch,
                        batch_size=batch_size,
                        shuffle=True,
                        validation_data=(X_test_AE, X_test_AE),
                        verbose=1,
                        callbacks=[checkpointer, tensorboard]).history
```

```
#Lets Load Model
autoencoder = load_model('model.h5')
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until

```
#plot the loss and Val loss
plt.plot(history['loss'])
plt.plot(history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper right');
```



```
## Lets predict on test and evaluate

predictions = autoencoder.predict(X_test_AE)
mse = np.mean(np.power(X_test_AE - predictions, 2), axis=1)
error_df = pd.DataFrame({'reconstruction_error': mse,
                        'true_class': y_test_AE})
error_df.describe()
```

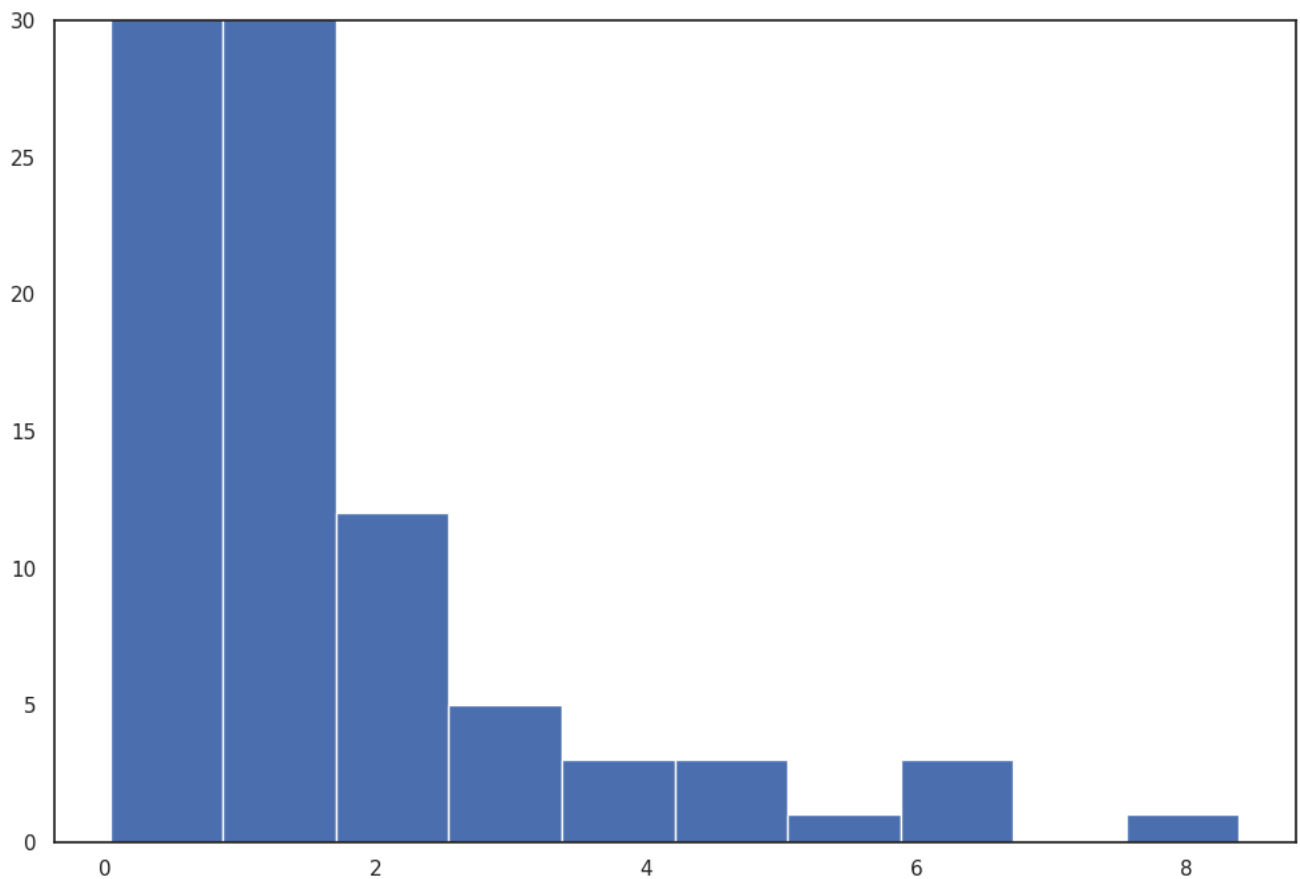
34/34 0s 4ms/step

	reconstruction_error	true_class
count	1082.000000	1082.000000
mean	1.603512	0.097043
std	11.193551	0.296153
min	0.039661	0.000000
25%	0.259826	0.000000
50%	0.441503	0.000000
75%	0.580475	0.000000
max	227.407266	1.000000

```
#Reconstruction error for Normal data
fig = plt.figure()
ax = fig.add_subplot(111)
normal_error_df = error_df[(error_df['true_class']== 0) & (error_df
['reconstruction_error'] < 10)]
_ = ax.hist(normal_error_df.reconstruction_error.values, bins=10)

ax.set_ylim(0,30)
```

(0.0, 30.0)



```
## Reconstruction Error with Fraud data

fig = plt.figure()
ax = fig.add_subplot(111)
fraud_error_df = error_df[error_df['true_class'] == 1]
_ = ax.hist(fraud_error_df.reconstruction_error.values, bins=10)
ax.set_ylim(0,30)
```

```
(0.0, 30.0)
```